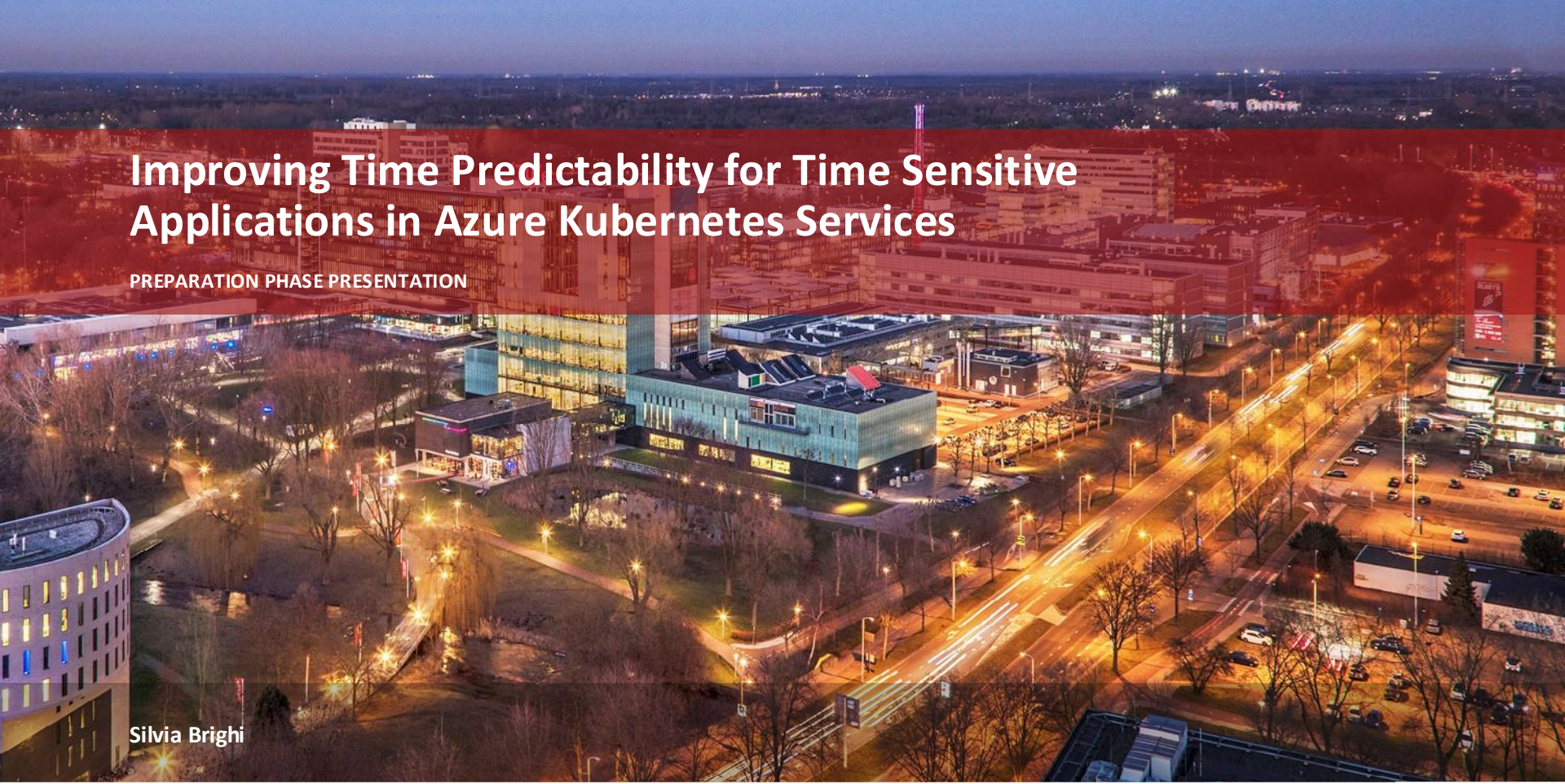
Graduation Project Updates

PREPARATION PHASE PRESENTATION

Silvia Brighi

Improving Time Predictability for Time Sensitive Applications in Azure Kubernetes Services

PREPARATION PHASE PRESENTATION



Silvia Brighi

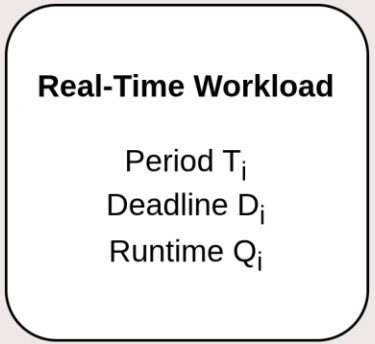
Interconnected Resource-aware Intelligent Systems - IRIS

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Time-Sensitive Applications

Time-sensitive application (TSA) are workloads whose correctness does not only depend on the outcome but also on when the outcome is produced.

A time-sensitive application are represented by:



Mayor shift of TSA to the cloud through containerization, but

- **Requires predictability**

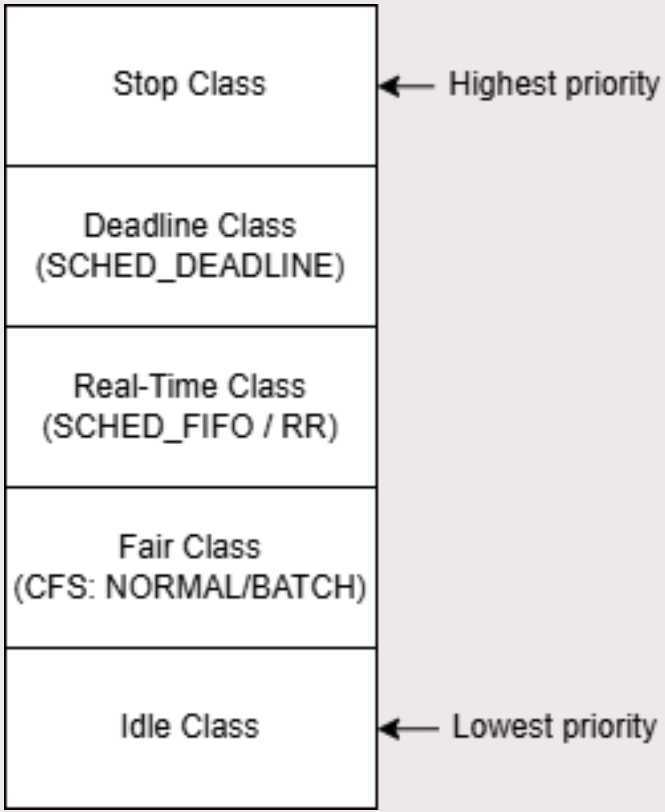
Linux Scheduling Foundation

Linux uses a **hierarchical scheduling class** structure.

In Linux, there are two real-time scheduling classes:

- **Deadline Class**
- **Real-time Class**

CFS is the **default scheduling class**:
- Allocates CPU time fairly
- no time guarantees for real-time workload



Why AKS Cannot Provide Real-Time Guarantees

Azure Kubernetes Service (AKS)

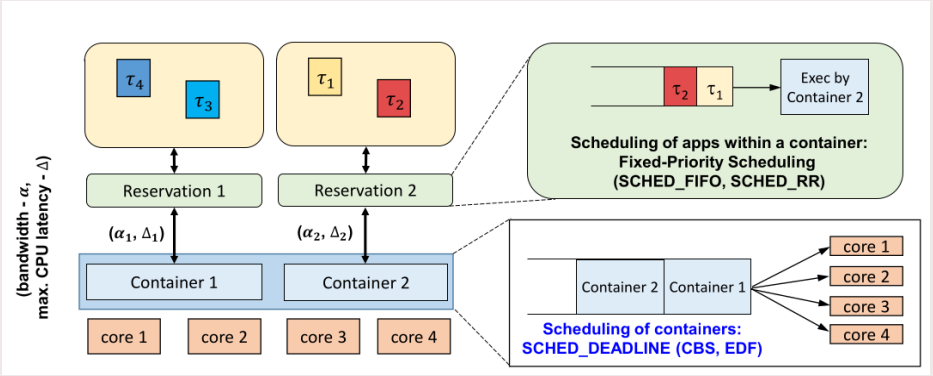
- Enables cloud-deployment of containers
- Azure manages infrastructure and control plane
- Kubernetes uses CFS scheduler

Interference Factors are:

- Multi-Tenancy
- Virtualization
- Node Churn
- Physical Placement

KubeDeadline for enabling real-time containerization

- Innovative solution for introducing with minimal changes to Kubernetes
- Allow scheduling for Pods with time constraints
- Provides custom version of the Dynamic Resource Allocator (DRA) and customize version of containerd and runc.



However, the patch was tested on **bare-metal, not in cloud environments**

Problem Statement

AKS is optimized for scalability, not predictability
→ Execution times are **highly variable**

Real-time techniques assume **stable environments**
→ These assumptions break in the cloud

Can we improve time predictability for time-sensitive applications running on Azure Kubernetes Services?

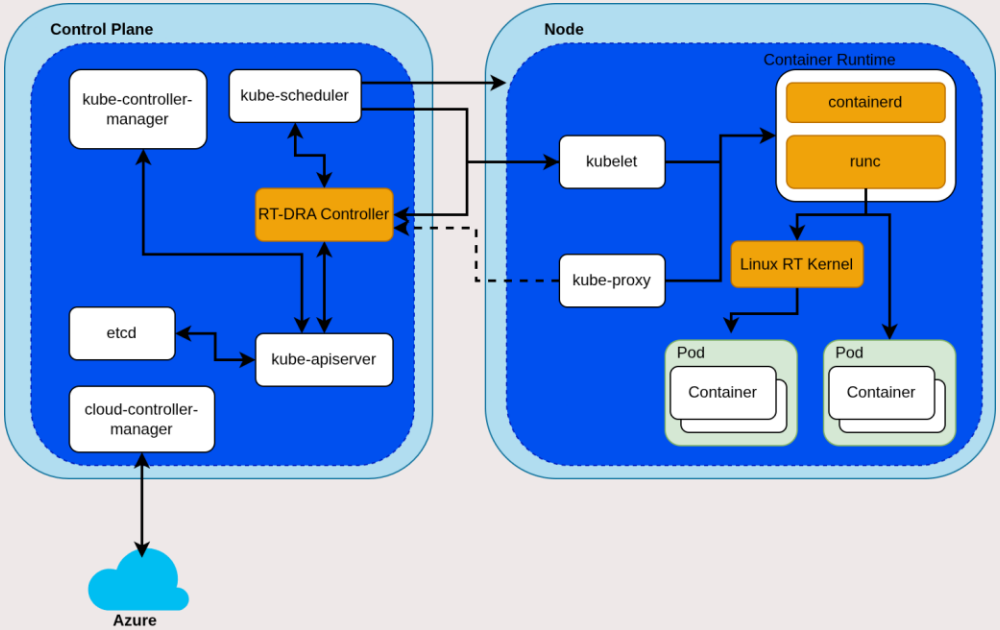
Research Questions

- **RQ 1**: Where do KubeDeadline's scheduling guarantees break down when deployed on AKS?
- **RQ 2**: How to assign parameters of KubeDeadline, such as budget, deadline and period, to time-sensitive tasks in a cloud environment where execution-time characteristics are non-deterministic?
- **RQ 3**: Which Azure IaaS configuration and infrastructure choices most significantly reduce execution-time variability for time-sensitive workloads running under KubeDeadline, and how should these be selected?

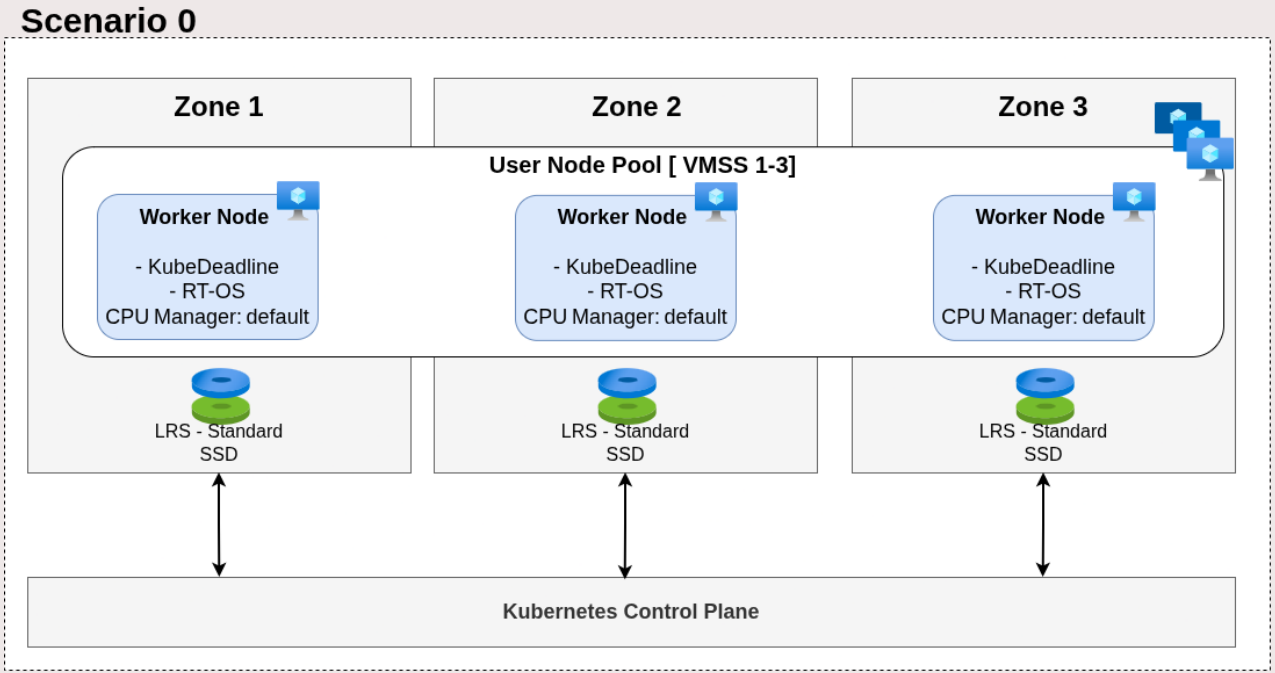
Create working prototype

Methodology is grounded on a prototype of AKS with KubeDeadline:

1. Use CAPZ instead of AKS
2. Build custom OS for VM
3. Introduce KubeDeadline



Prototype Architecture



RQ 1: Analyse impact of cloud influence on KubeDeadline guarantees

KubeDeadline assumes isolated environment, but
-> Public cloud adds extra interference

- **Controllable factors:** co-located workload intensity, CPU Manager policy, node pool isolation

- **Non-controllable factors:** hypervisor interference, CPU migration, cold cache

Synthetic periodic RT workload used to study impact on execution-time distribution

RQ2: Parameter derivation for KubeDeadline

Provide a procedure to derive KubeDeadline parameters budget, period and deadline

We will investigate the **measurement-based** approaches:

1. Stress Profiling
2. Tail Estimation
3. Inflation Factor Estimation

The final procedure will be defined **after** analyzing experimental results.

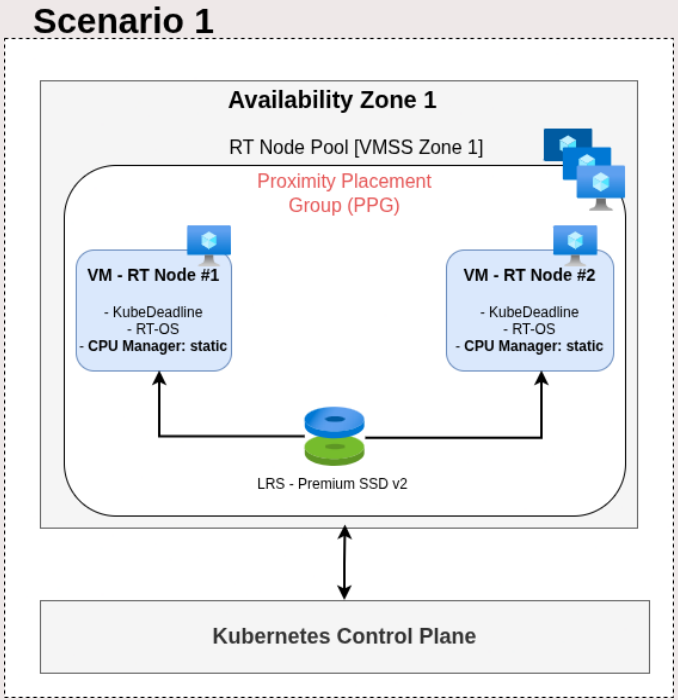
RQ3: Azure Specific configurations for time-sensitive applications

Baseline: implemented prototype

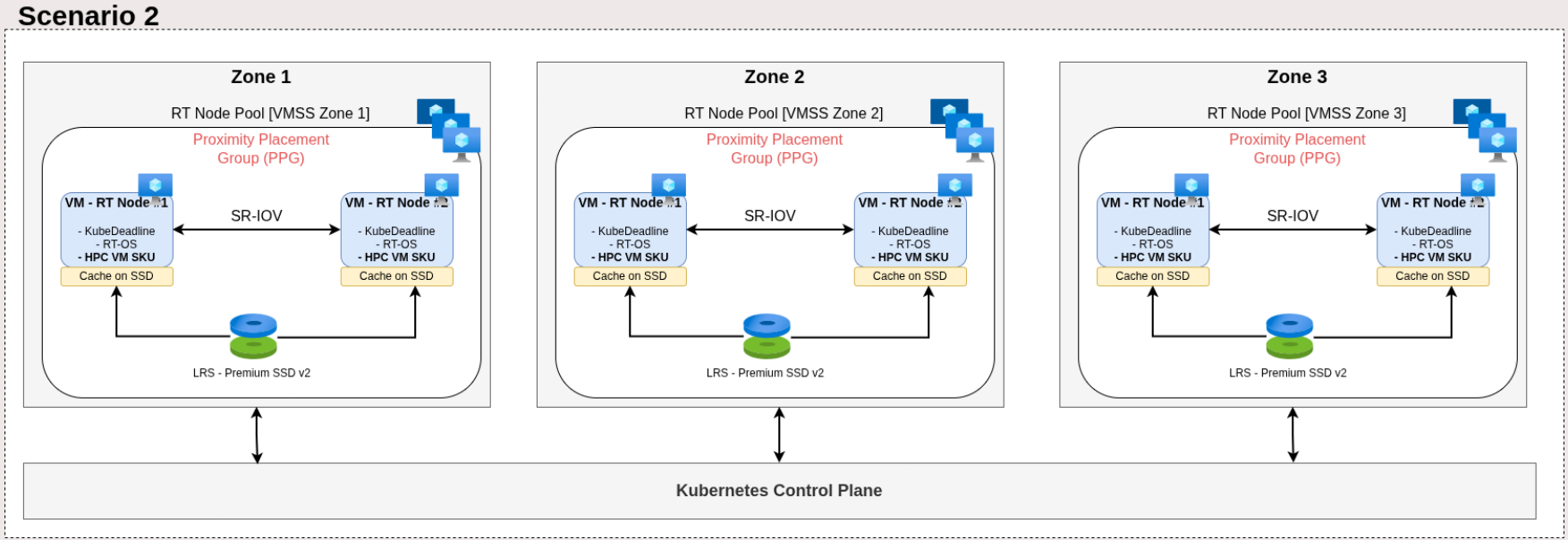
Compare with optimised configurations:

- **Scenario 1**: Department-level
- **Scenario 2**: Enterprise-level

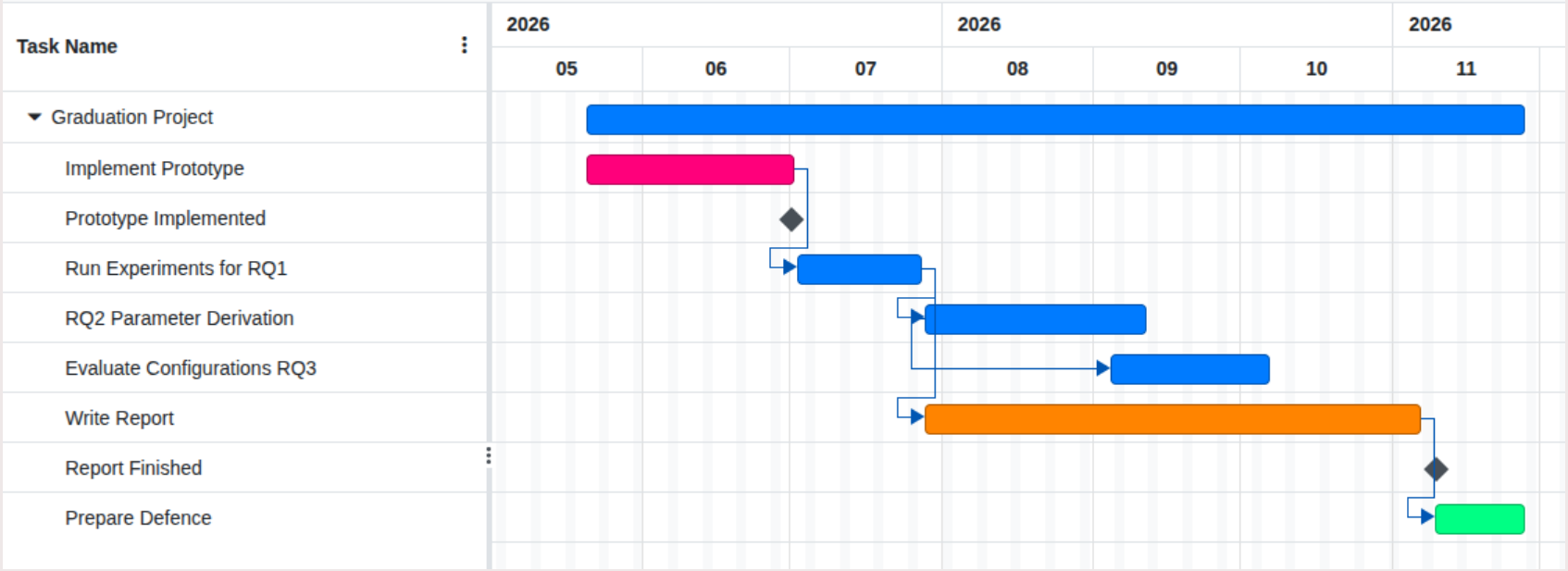
Scenario 1 – Departmental configuration



Scenario 2 – Extended Configuration



Planning


Task Name
2026
2026
2026
05
06
07
08
09
10
11
Graduation Project
Implement Prototype
Prototype Implemented
Run Experiments for RQ1
RQ2 Parameter Derivation
Evaluate Configurations RQ3
Write Report
Report Finished
Prepare Defence

Phase 1: have KubeDeadline Cluster working

The first goal of the project is to have a K8s cluster with RT-DRA installed and running as expected. This means:

- **Containers** are assigned correctly to the cgroups with the correct parameters:
  - cpu.rt_runtime_us (cpu.rt_multi_runtime_us)
  - cpu.rt_period_us

  And the reservation of the containers works correctly
- **Tasks** inside the containers can be correctly scheduled with RT class (SCHED_FIFO)

Problem encountered

KubeDeadline framework heavily relied on old and deprecated versions of Kubeadm, containerd, runc and H-CSB kernel.

Therefore:

- H-CBS kernel had a different admission control test
- H-CBS was lacking the cpu.rt_multi_runtime_us parameter
- RT-DRA was unable to write the CDI device file on the worker node
- RT-DRA was unable to pass correctly the reservation parameters
- Runc could not properly set reservation parameters to the container

The fixes

KubeDeadline works now for the kernel rt-cgroups-multi-260514

RT-DRA driver:

- Fixed the CDI device naming so that containerd can parse the device file correctly

RT-runc:

- Correct the write order: write cpu.rt_period_us and then cpu.rt_period_us
- removed old references to cpu.rt_multi_runtime_us (now unified in cpu.rt_runtime_us with per-core runtime list <runtime> <cpu> <runtime> <cpu>
- Seed ancestors: kubepods.slice -> kubepods.<QoS>.slice only if they read 0 because the kernel requires: sum(children) <= parent

Now the cluster works correctly

```
RT-DRA NODE-SIDE EVIDENCE
generated : 2026-07-01T10:37:26+00:00
node      : rt-cluster-worker-0
kernel -r : 7.0.0+
kernel -v : #3 SMP PREEMPT Mon Jun 22 18:00:14 UTC 2026
cgroup fs : cgroup2fs (cgroup2fs => unified v2)
RT_GROUP_SCHED : CONFIG_RT_GROUP_SCHED=y
============================================================
[PROOF A] Does the driver's target file 'cpu.rt_multi_runtime_us' exist in /sys/fs/cgroup?
------------------------------------------------------------
  RESULT: 0 occurrences. 'cpu.rt_multi_runtime_us' is ABSENT from the entire cgroup tree.
  => Every write the driver does to this filename has NO target (silently lost).
------------------------------------------------------------
[PROOF B] Which cpu.rt_* files DOES the kernel expose? (root of /sys/fs/cgroup)
------------------------------------------------------------
  cpu.rt_period_us = 1000000
  cpu.rt_runtime_us = 950000
  NOTE: presence of cpu.rt_runtime_us under cgroup2fs proves this kernel
        exposes an RT interface on v2 -- the gap is the MULTI file, not v1/v2.
------------------------------------------------------------
[PROOF C] RT pod budget chain (cpuset pinned, but rt_runtime_us all zero?)
------------------------------------------------------------
  online CPUs on node: 0-3
  candidate leaf scope(s): 1
  LEAF: /sys/fs/cgroup/kubepods.slice/kubepods-besteffort.slice/kubepods-besteffort-pod6958f966_511c_4f81_a3e5_ad4da22aa73a.slice/cri-containerd-13ff8a8d3b001490
1839fee7352346cc48b448bdefcef00a4143b5c6b32a3aad.scope
    cri-containerd-13ff8a8d3b0014901839fee7352346cc48b448bdefcef00a4143b5c6b32a3aad.scope rt_runtime_us=100 0 0 0    rt_period_us=1000       cpuset.cpus=0
    kubepods-besteffort-pod6958f966_511c_4f81_a3e5_ad4da22aa73a.slice      rt_runtime_us=100 0 0 0    rt_period_us=1000       cpuset.cpus=
    kubepods-besteffort.slice                                             rt_runtime_us=950000       rt_period_us=1000000    cpuset.cpus=
    kubepods.slice                                                        rt_runtime_us=950000       rt_period_us=1000000    cpuset.cpus=
    cgroup                                                                rt_runtime_us=950000       rt_period_us=1000000    cpuset.cpus=(absent)
  [PROOF D] scheduler policy of tasks in this leaf:
      pid=12178 comm=bash policy=SCHED_OTHER (CFS)
      pid=12457 comm=sleep policy=SCHED_OTHER (CFS)
      pid=12926 comm=sleep policy=SCHED_FIFO (RT)
```

Phase 2: KubeDealine guarantees in the cloud

The goal is to analyse the impact of public cloud interference on KubeDeadline guarantees, namely:

- **Timing requirement**: tasks deadline
- **Delta Δ**: maximum delay in the provisioning of the reservation
- **Alpha α**: guaranteed bandwidth

When do these guarantees break?

Plan to reach the goal

The idea is to use period workloads that are guaranteed to be schedulable according to CARTS and multi-processor scheduling:

1. Generate synthetics periodic task sets with U between 1.4 and 1.8
2. Run the task sets with Vanilla K8s and KubeDeadline
3. Planned measurements to retrieve:
    1. Execution time
    2. Response time
    3. Deadline misses

    To derive
    - the distribution curves of execution/response time
    - Avg/media/max of execution/response time for every utilization percentage
    - Cumulative Distribution Curve for Deadline miss ratio on utilization

Variability in the results

Give the highly variability in the results, the plan is to execute the experiments:

- Different times of the day
- Different days

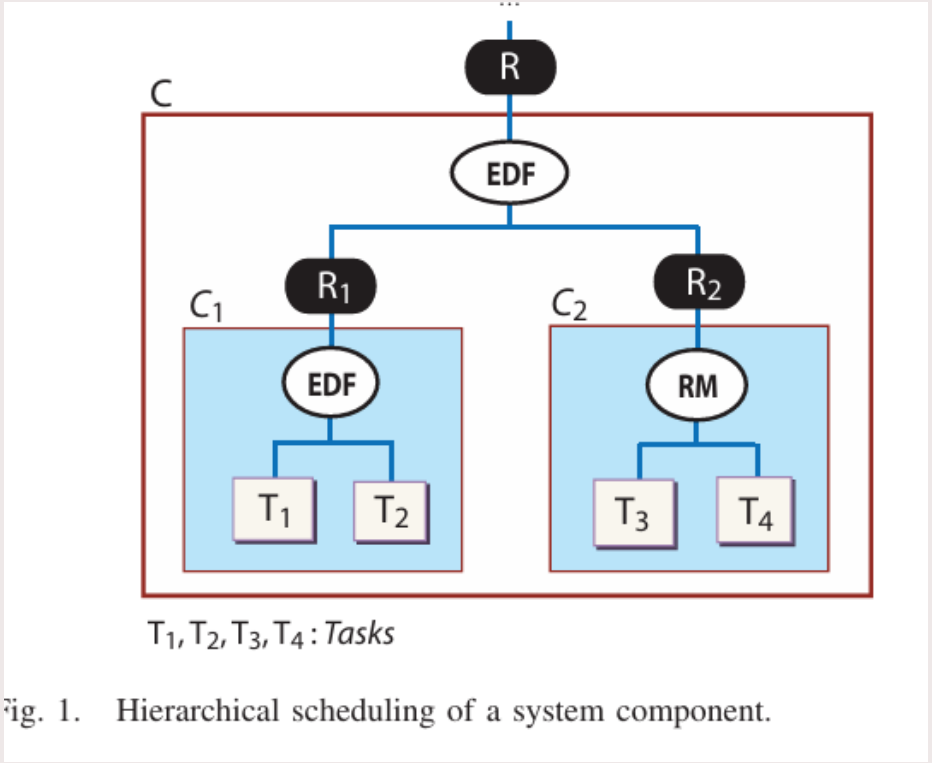
To catch congestion and noisy neighbours. Possibly by repeating the experiments 5/10 times per (time, day)

Idea is to collect enough evidences to highlight the variance in the execution/response times

What is CARTS?

CARTS (Compositional Analysis of Real-Time Systems) is a tool that allows the generation of resource interfaces needed for the compositional analysis of RTS. Supports

- standard schedulers such as Rate Monotonic and Earliest Deadline First
- and can generate both periodic and explicit deadline periodic resource interfaces.



ig. 1. Hierarchical scheduling of a system component.

Source: CARTS: A Tool for Compositional Analysis of Real-Time Systems, Phan et al 2011

RQ1 – Experimental Setup

The workload

One SCHED_FIFO task inside one SCHED_DEADLINE / H-CBS
container reservation → Q = budget, P = period, m = 1 core

Two-level view

Container = server → delivers bandwidth $\alpha = Q/P$
with bounded delay $\Delta \approx 2(P-Q)$
Task = demand → execution time C, response time R

Node

Azure D4s_v5 = 4 vCPU = 2 physical cores (hyper-threaded)

Workloads have two categories:

- Tight P ≈ 10 ms
- Soft P ≈ 100 ms

Execution-layer breaks are period-independent
→ persist at BOTH scales
Delay-layer breaks are absolute-time events
→ hurt at 10 ms, fade at 100 ms

0- Baseline

Goal

Establish the reference response-time distribution when “1 vCPU = 1 dedicated core” is genuinely true.
Measure the irreducible virtualization noise floor (hypervisor steal) with zero guest contention.

Both: tight 10 ms & soft 100 ms
U swept 0.1 → 0.94, one physical core, hyper-thread sibling taken offline.

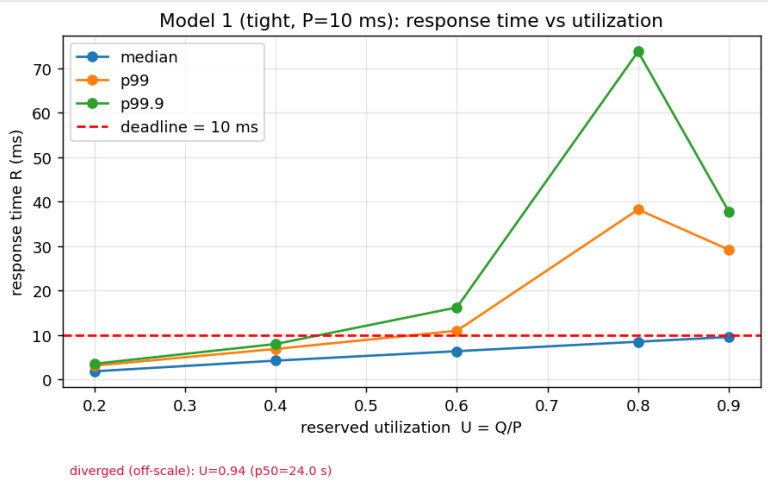
Metrics

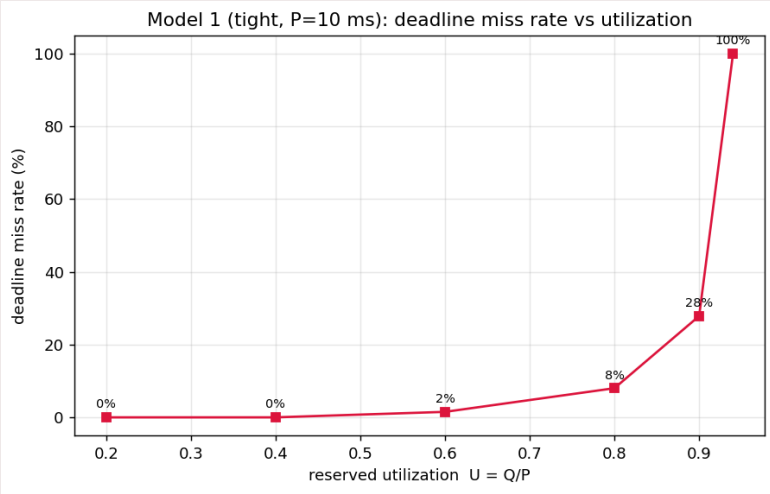
Response Time and Execution Time distribution incl. tail (p99.9, max)
→ defines “clean” for every other model
Steal time + IRQ time
→ these can be the only host effect that can show deviations

Expectation

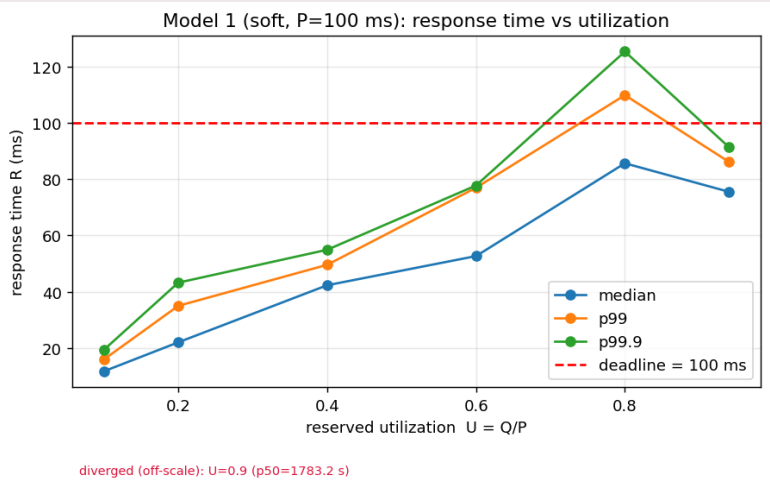
Tight, clean R ≈ C, no deadline misses at both scales.
Any deviation here already = irreducible steal-time noise.

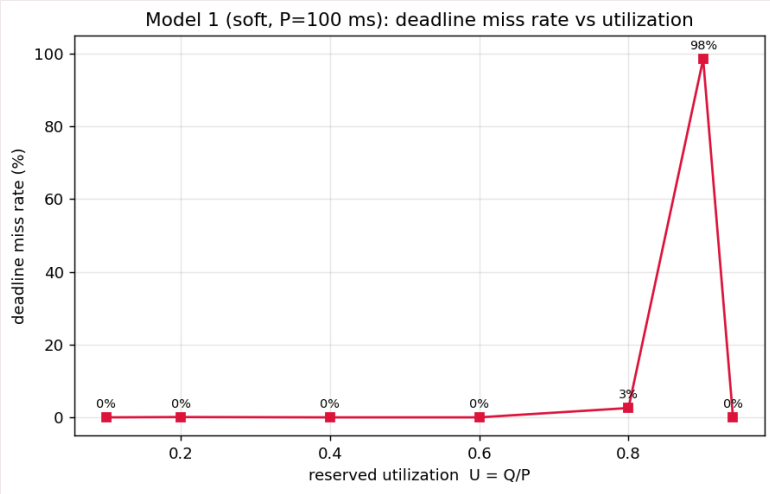
Results obtained from Model 1 – tens of ms





Results obtained from Model 1 – hundred of ms





1- Co-located neighbours

Goal

Does the CBS reservation constraints the RT container from noisy/interference pods in the SAME VM?
Idea is to tests the isolation the theory promises (shared guest kernel CFS / CBS hierarchy).

Both: tight 10 ms & soft 100 ms
U swept 0.1 → 0.94, one physical core, hyper-thread sibling taken offline.

Metrics

Response/Execution time & α_eff vs neighbour intensity
→ flat line = isolation holds
Δ_eff, deadline-miss rate
→ catches jitter / bandwidth leaks

Expectation

If guarantee holds: flat vs load, both scales.
Degradation with load = leak.
Persists at soft → bandwidth leak;
fades at soft → latency/jitter leak.

2- Hyper-thread vs physical pinning

Goal

Kernel admits ≈3.8 “cores” but only 2 exist.
Above ~2.0 U you are forced onto HT siblings.
Show the break appears as C-inflation, not lost bandwidth.

Both scales: same job on a clean physical core vs a busy HT sibling.
HT contention is period-independent.

Metrics

Execution time C (physical vs sibling)
→ the direct C-inflation signal
Signature: server got Q (α,Δ intact)
but task still missed → provisioning

Expectation

Break at BOTH scales, similar relative size
→ identifies it as execution-layer.
It is the Direct input to RQ2 (recalibration).

3- IRQ steering

Goal

Interrupts steered ONTO vs OFF the RT core.
Show interrupt affinity is a controllable determinant of real-time jitter in the cloud; quantify bounded vs heavy-tailed impact.

A fixed-µs IRQ burst is large vs 10 ms, negligible vs 100 ms.

Metrics

Per-core IRQ time × R-tail correlation
→ positive attribution of the tail
Δ_eff / replenishment delay
→ shows budget arriving late

Expectation

Off-core → tail collapses to M1 baseline.
On-core → tail inflates at 10 ms, fades at 100 ms → delay-layer.
Mirror image of Model 3.

A new error was found…

I've started noticing that every time I was running a pod, this was failing with a StartError.
The problem -> in the cgroup tree, rt_runtime_us remained set to 0 and new tasks/pods could not ask to get set a runtime_us higher than the parent.

Therefore, no pod was starting.

How to fix it?
My workaround: set sched_rt_runtime_us to -1, then seed the children starting from the root /proc/sys/fs/cgroups

However, -1 removes completely any admission control on the DL server and so allows to set whatever value to the childer.

This is wrong because it won't throttle any RT task and
ad to starvation

But why is this happening?
By nature, these rt parameters are represented in the kernel as filesystem and everytime there is a reboot these are deleted.
So, it is normal that /sys/fs/cgroup is empty (set to 0) at reboot, but when k8s files are created at boot these should be seeded too. And this does not happen.

Why?

Workload of the experiments

Instead of using a busy loop as workload, I'm going to use a fixed double-precision matrix multiply with C = A x B and with size MxM with M default is 48.
A/B/C are tuned so that they can fit in L1/L2

However, how can we have a deterministic C (execution time)?
A and B are filled only once from a fixed seed and reused for every job. IN this way we can guarantee a data-independent control flow, tiny cache and no allocations/syscalls/I/o in the hot loop

+ the per job compute demand is tuned only by the repetition count K (the rep of the same matmul) and never by the data

-> Execution time has near-zero intrinsic variane (the target for the fine tuning was CV < 2%

The calibration for K

The question is for each experiment cell, how many reps K make the probe demand exactly its reservation?

The grid (cells). Each cell is a (scale, U) pair:

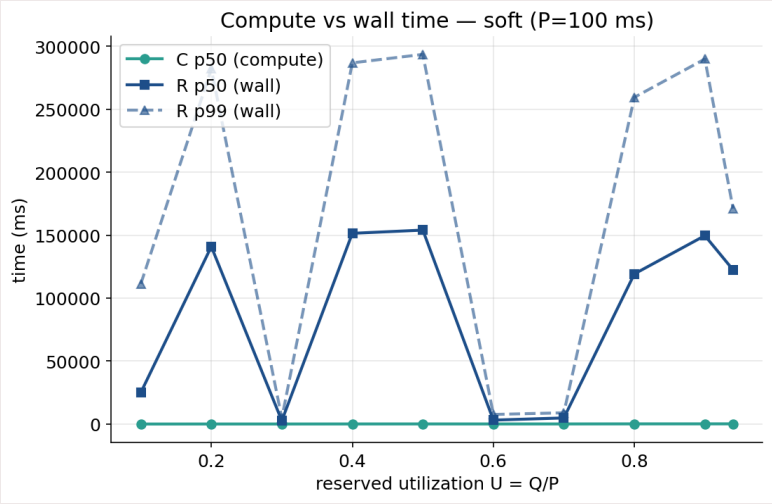
Scales: tight P = 10 ms, soft P = 100 ms.
Utilization U ∈ {0.1 … 0.9, 0.94} (0.95+ is refused admission by the rt-DRA driver).
The budget is derived, never hardcoded: Q_us = round(U · P_us).
The goal. Find K so the median per-job execution time C ≈ Q = round(U·P). Then the probe demands exactly its reservation, so a deadline miss means the environment failed to deliver bandwidth. not that the job was too big.
1- Runs on an isolated physical core, either in a calibration pod on the real experiment node (kubectl exec + taskset pin) .
2- Because C is linear in K (K reps of the same fixed matmul), it doesn't do a blind binary search. It seeds K, measures the slope c1 = C/K, then solves K = Q / c1 and refines in a few probes.
3- For each cell it records K, median_C_us, and the coefficient of variation (CV). A clean, low-CV C is a precondition for delay attribution, so if CV exceeds kernel.cv_threshold (2%), it fails loudly (exit 3) It requires to to investigate sibling activity, lack of isolation, or DVFS before trusting any results.

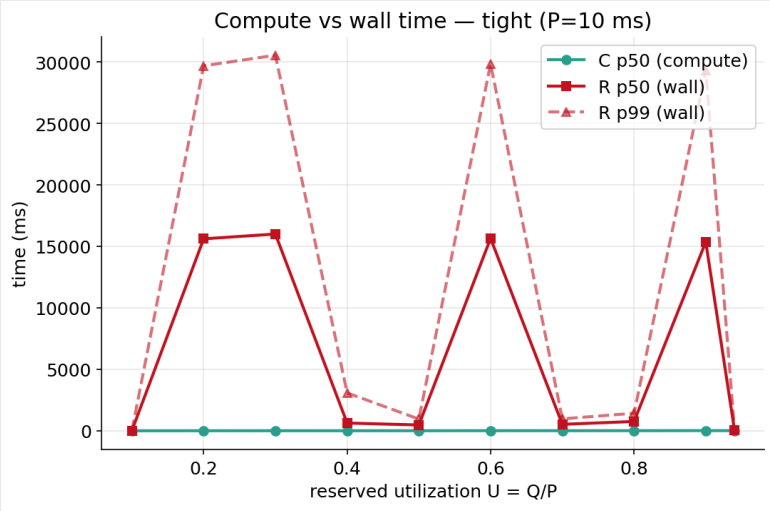
The calibration

- Low-Q tight cells (U0.1–0.6, Q = 1–6 ms) and a couple of short soft cells are the noisy ones; **mid/high-Q cells are clean** (CV ~1%).
- Calibration ran **on cpu0** — the busiest core (system IRQs, softirqs, kubelet, your own sampler/node-prep DaemonSets, SMT sibling on cpu1). It's **not actually isolated**, so short jobs pick up scheduling/IRQ/DVFS jitter.
- Short jobs amplify it: a fixed ~30–50 µs of jitter is a big *fraction* of a 1 ms job → high CV at low Q, negligible at high Q. That's why the ceiling cells (soft-U0.9/0.94) are clean (CV <1%) but tight-U0.1 isn't.

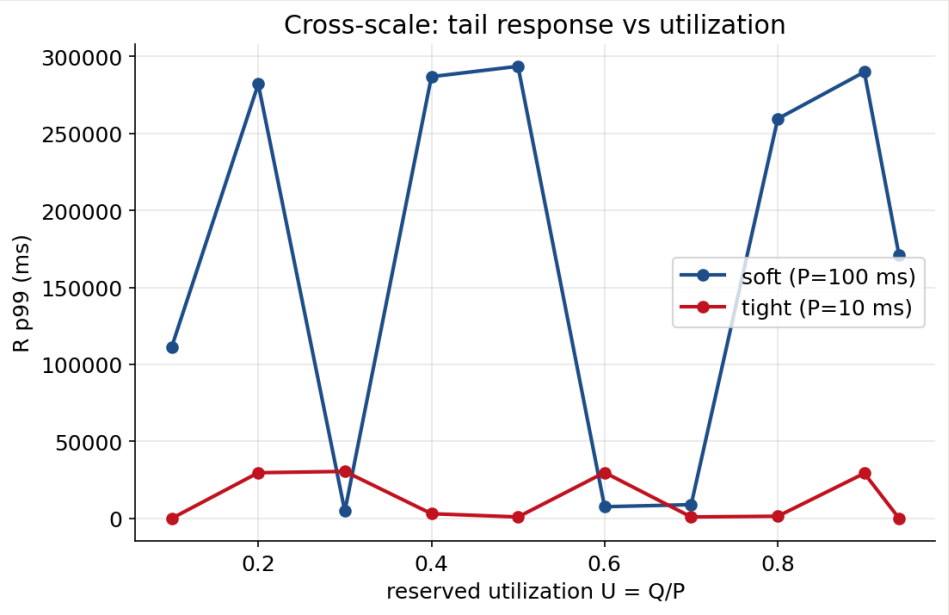
This is a cloud-VM limitation, not a bug — and it's a legitimate preliminary result: *"on an Azure D4s_v5 the 'isolated' RT core isn't clean; C-CV is 3–18% at low utilisation, which limits delay attribution at the tight/low-Q corner."*

Some preliminary results – Model 1

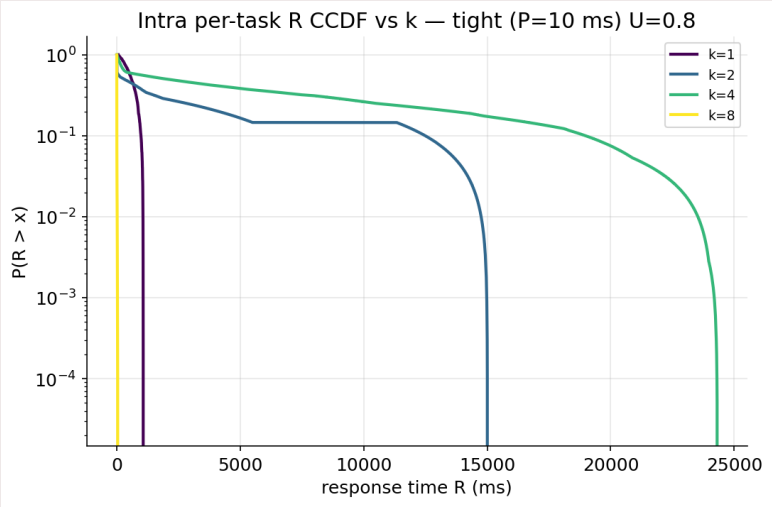


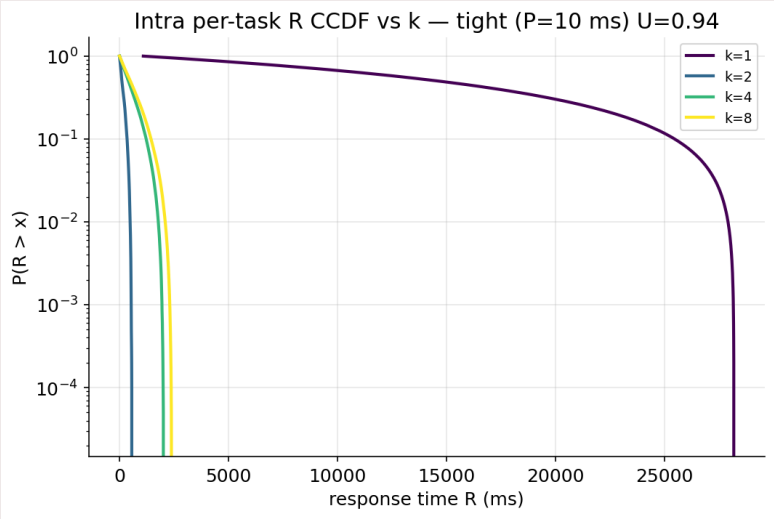


Some preliminary results – Model 1

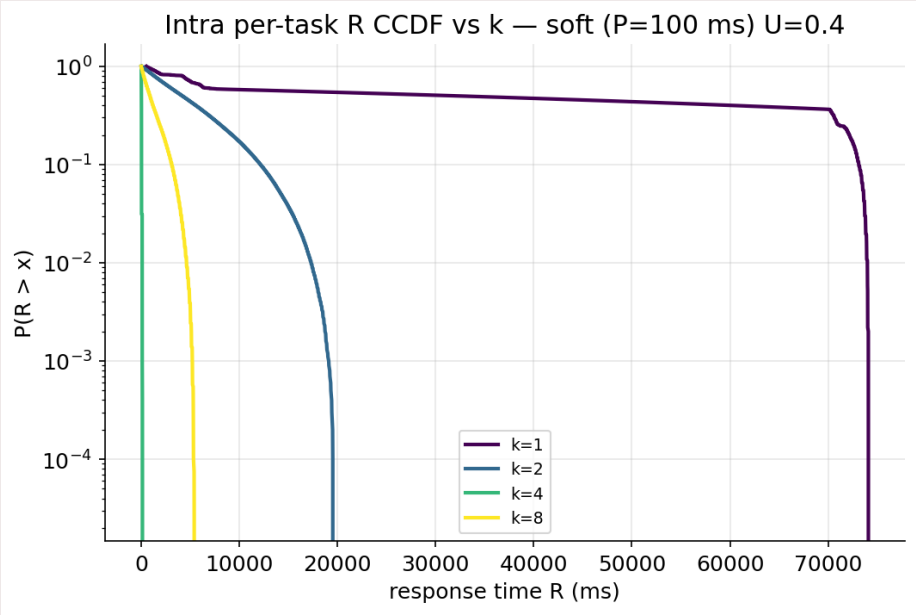


Model 2

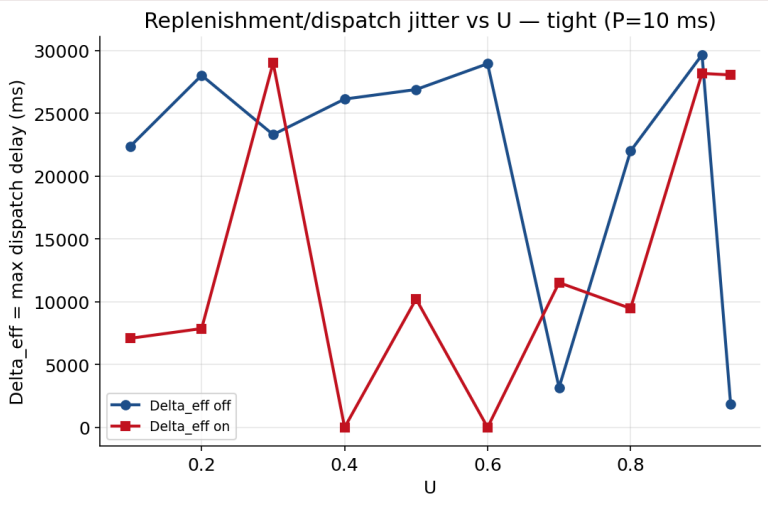

Intra per-task R CCDF vs k — tight (P=10 ms) U=0.8
P(R > x)
response time R (ms)
k=1
k=2
k=4
k=8


Intra per-task R CCDF vs k — tight (P=10 ms) U=0.94
P(R > x)
response time R (ms)
k=1
k=2
k=4
k=8

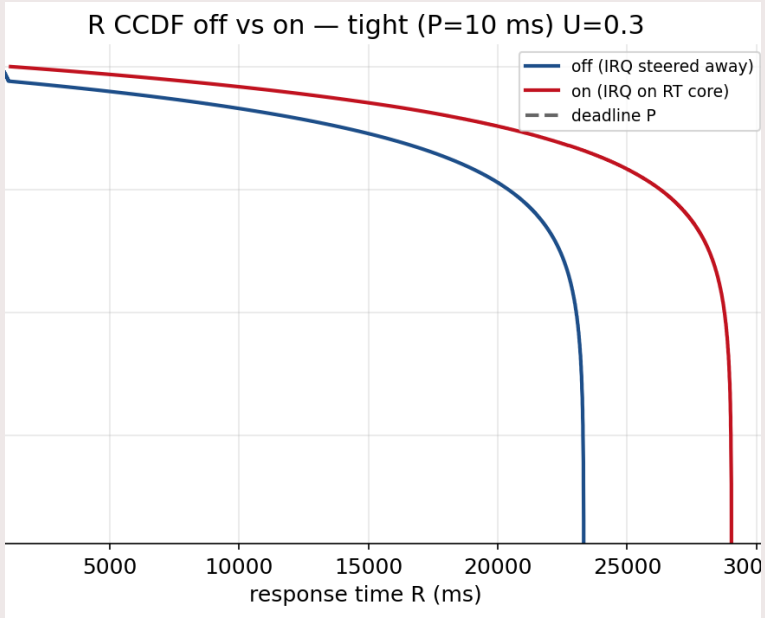
Model 2

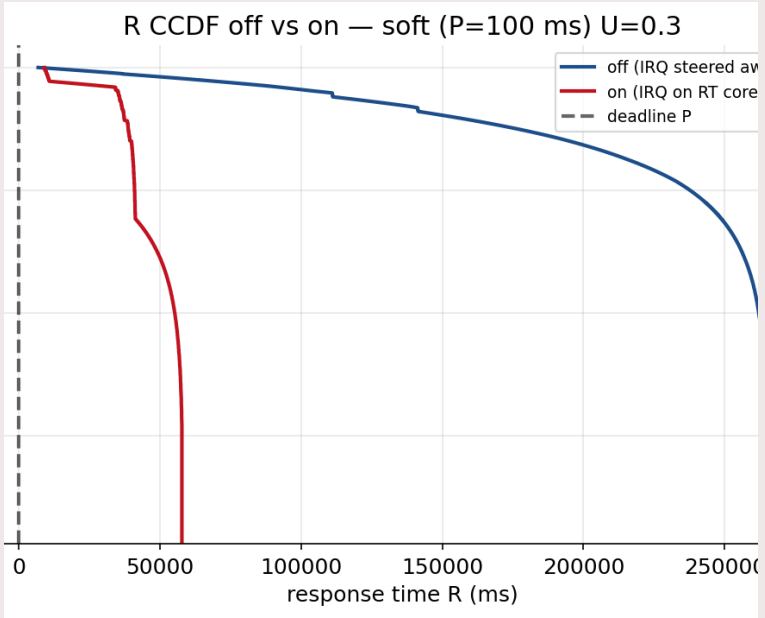

Intra per-task R CCDF vs k — soft (P=100 ms) U=0.4
P(R > x)
response time R (ms)
k=1
k=2
k=4
k=8

Model 4


Replenishment/dispatch jitter vs U — tight (P=10 ms)
Delta_eff = max dispatch delay (ms)
30000
25000
20000
15000
10000
5000
0
0.2
0.4
0.6
0.8
U
Delta_eff off
Delta_eff on

Model 4


R CCDF off vs on — tight (P=10 ms) U=0.3
off (IRQ steered away)
on (IRQ on RT core)
deadline P
5000
10000
15000
20000
25000
response time R (ms)


R CCDF off vs on — soft (P=100 ms) U=0.3
off (IRQ steered aw
on (IRQ on RT core
deadline P
0
50000
100000
150000
200000
response time R (ms)

Why do we see this behavior?

Because of the Turbo Boost / DVFS

The frequency-driven sweep confound is a general DVFS artifact, not a cloud phenomenon; it must be controlled on any platform (hence frequency pinning).
What is cloud-specific is
1- that the guest may be unable to control or even observe its true frequency, and
2- the presence of steal time and host-side interference that have no bare-metal/laptop equivalent

DVFS makes each job's execution time larger -> between turbo and basic there can be roughly ~1.2-1.5x

If the execution time amplifies for such term, then R amplifies at a higher magnitude for two reasons:
1- Accumulation: the small per-job overrun (C-Q) stack up and becomes over all the jobs second of backlog
2- change from bounded to unbounded: because the calibrated is $C \approx Q$ with near-zero headroom, every small change in C when C = Q, leads to an uncontrolled R

Past weeks focus was stability in the experiments:

At the beginning, the collected data and its numbers weren't behaving the way the underlying theory predicted. Miss rates that should have tracked utilization smoothly were jumping around unpredictably, and repeated runs of the same experiment sometimes told different stories...

The larger source of instability was also **OS preemptions, background housekeeping tasks**, that turned out injecting variability that had nothing to do with the actual research question. That distinction mattered, because until we could tell a real effect apart from an artifact of an uncontrolled setup, we couldn't trust any conclusion we drew from the data.

So before pushing forward on new results, Ive spent the past weeks tracking down exactly where that instability was coming from and closing it off, one source at a time.

Stabilizing the node -- what we actually control and what not

I've isolated the physical cores used for measurement from the rest of the Linux kernel's own housekeeping. Concretely,

- enabling isolcpus so the scheduler never places unrelated processes on those cores,
- Enabled nohz_full so the periodic timer tick stops firing there,
- Enabled rcu_nocbs so RCU callback processing is handled elsewhere instead of stealing cycles from the isolated core.

CPU frequency was pinned to a fixed governor, so frequency scaling (DVFS) itself doesn't become a source of timing variation. One core is deliberately left out of isolation on purpose (cpu0), so the OS and Kubernetes still have somewhere to do their own necessary work without that work leaking onto the cores we are actually measuring.

Isolcpus: Removes these cores from the scheduler's normal pool, so nothing gets placed on them except what we explicitly pin there ourselves.
nohz_full: Stops the periodic timer tick on the isolated cores, so the kernel is not interrupting our measurement just to do routine bookkeeping.
rcu_nocbs: Moves memory-cleanup (RCU) processing off the isolated cores onto a separate core, so that work never pauses a measurement in progress.
Fixed frequency governor: Locks the CPU at a fixed, maximum clock speed, so the same code cannot run at different speeds from one run to the next.

Where this stops working:

All of this happens inside the guest VM and that is also its limit.

On Azure IaaS, we have no visibility into what the hypervisor itself is doing underneath: whether our isolated core is a dedicated physical core for real, whether another tenant is sharing that same physical hardware or whether the hypervisor briefly steals CPU time from our VM to schedule someone else.

Isolation inside the guest stops the guest's own kernel from interfering with our measurements, but it cannot stop interference that happens one layer below, where we simply have no view. What is left after all of this isolation is a small residual noise floor (coefficient of variation hopefully lower than 5%) that cannot be attributed to anything inside our control, and we should see that a possible lower bound of what this setup can achieve on shared cloud infrastructure.

New version of the dra-rt-driver

I'm finishing a new version of the driver to introduce determinism in the choice of the cores. By default, the choice was **Worst Fit**, with the possibility for the user to use also **Best Fit,** but everything was hard coded.
This introduced the assign-get core-retry mechanism in the experiments which is not optimal and increases time.

The new version introduces a new parameter in the manifest together with those ones that are already present:

RequestedCpus : int[]

Which contains the ID of the cores a user wants for the pods.

Those cores are checked at the moment of the assignment to see whether they are available, have enough utilization available for the workload (the sum of the utilization + the utilization of the workload should be less than 95%)

On your questions: Is the data identical, and what kind of variability are we actually measuring

The experiments use two probe workloads:

1. a compute bound one that performs dense matrix multiplication with its working set kept inside cache,
2. a memory bound one that chases pointers through a buffer larger than the last level cache, deliberately defeating the hardware prefetcher.

Every probe run uses a fixed seed. The matrices in the compute-bound workload and the memory buffer layout in the memory-bound workload are both deterministic from that seed, identical across every job, every cell, every round we have collected. So yes, the data is identical, not only the instruction sequence.

Given that, everything we have collected so far measures internal variability: the exact same execution, repeated many times, under different levels of contention. We have not yet collected data on external variability, meaning the same execution with different input data.

Given that, everything we have collected so far measures internal variability: the exact same execution, repeated many times, under different levels of contention. We have not yet collected data on external variability, meaning the same execution with different input data.

Is that something I should consider to do?

The models, what each one tests, and where the data currently stands

Evry model was executed with each of the two workloads previously show.

- **Model 1.** Solo baseline, no co-tenant at all. Four rounds each of both workload types, both scales.
- **Model 3**. A 2 by 2 design (Four variations in total): the target shares a physical core either with its SMT sibling or with a thread on a separate physical core, and the co-tenant is either itself reserved or an ordinary unbounded process. The physical placement control arms are solid, both workload types, all four rounds. The sibling plus unreserved arm, the one thatmost directly tests where the guarantee actually breaks.
- **Model 4**. No external co-tenant at all. A single reservation whose own two threads run concurrently on two separate physical cores. Tests whether a reservation can still expose the same kind of gap purely from its own internal concurrency, this time via shared memory bandwidth rather than shared execution ports. Still collecting data but using the new version of the dra.

Model 3 design

	Sibling (Hyperthreaded) Cores (SMT)	Physical Cores
Interference workload with CFS	Model 3: runs on SMT with one container with CFS and the other with CBS	Model 3-w3: runs on physical cores with one container with CFS and the other with CBS
Interference workload with CBS	Model 3-w2: runs on SMT with b	Model 3-w4: runs on physical cores with both containers running with CBS

Claims, expectations, and results so far

Claims	Expectations	Status
0: cloud noise floor	Small, non-zero, distinguishable from real contention effects	Coefficient of variation around 0.02 to 0.04 at tight scale in the solo baseline, an order of magnitude smaller than the degradation seen under real contention
1: does reservation alone protect against SMT sharing	Two properly reserved workloads should still interfere if they share a physical core, since admission accounting works per logical core, not per physical core	Model2 with CFS and CBS interference workloads on SMT cores. Even with a fully reserved, properly admission-controlled co-tenant, execution time still gets pushed over budget in one of the four collected rounds, producing close to a 25 percent miss rate at the affected utilization levels once pooled across rounds. The admission gap is real even when both sides are well behaved

claim	Expectations	status
2: decomposing where guarantees break down	Physical separation should look clean regardless of co-tenant type. Sharing a physical core should degrade performance, and more so when the co-tenant is unbounded	Model3: Confirmed for the physical arms, clean across the entire tested range for both workload types. Confirmed for the sibling arms too, but with a specific mechanism.
3: severity of the deadline miss and not only its existence and quantity	A given miss rate could come from scattered isolated misses or from concentrated bursts, and these mean different things in practice	Model3: Confirmed as concentrated bursts. Misses cluster precisely at the utilization levels where execution time crosses the reservation budget, not scattered randomly
4: an actionable utilization threshold	Physical separation should hold guarantees across the full range tested. Sibling sharing should start breaking down at some identifiable utilization	Physical arms hold across the entire tested range, up to 94 percent utilization, both workload types. Sibling arm with the compute bound workload starts breaking down as early as 20 to 30 percent utilization at the soft scale, confirmed across four independent rounds, not a single-round estimate anymore

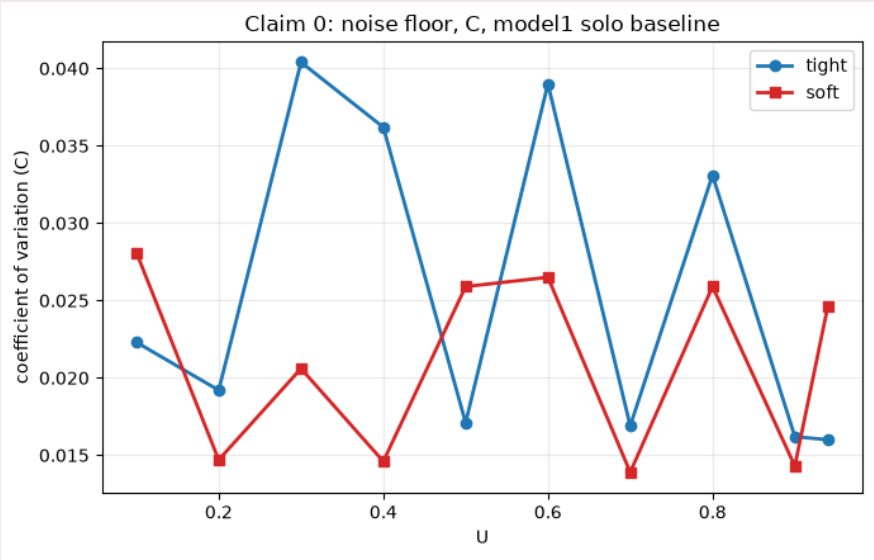
Claim	Expectations	Status
5: does the workload's own resource profile matter	A compute bound co-tenant should degrade the target far more than a memory bound one, because they compete for different hardware resources	Confirmed clearly. Compute bound co-tenants inflate execution time by roughly 1.8 to 2.1 times. Memory bound co-tenants inflate it by roughly 1.0 to 1.1 times, essentially negligible, under the identical lacement and contention setup
6: does a reservation's own concurrency expose the same gap	A single reservation's two threads should show self interference through shared memory bandwidth, more so for the memory bound workload than the compute bound one, since that concurrency expose the same	Data collection in progress

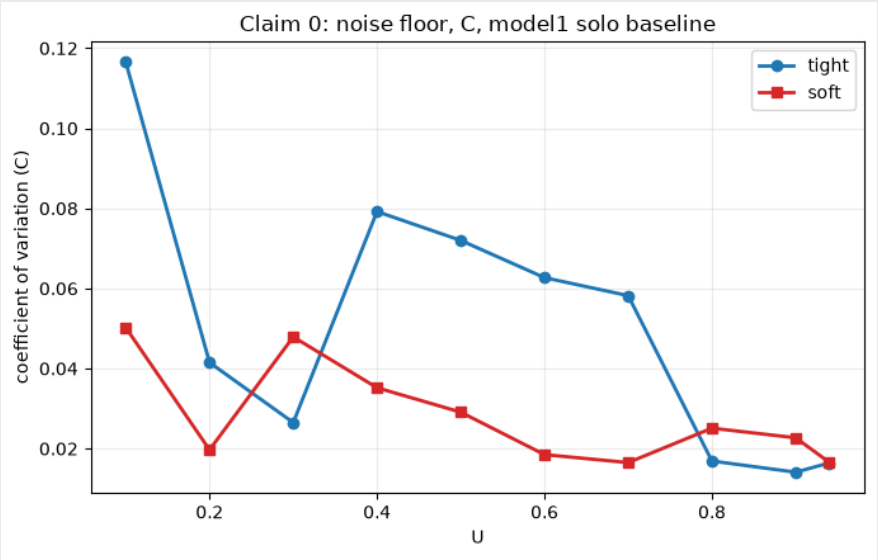
The idea behind claims 2, 3 and 4

The reservation gives a workload a fixed amount of execution time per period. Once that budget is used up, the kernel stops the workload until the next period begins. This means the relationship between execution time and its budget is not gradual, it is more like a cliff. As long as execution time stays under budget, the workload finishes comfortably and meets its deadline almost always.
But te moment contention pushes execution time just over budget, the workload gets suspended and has to wait for the next period: its response time roughly doubles, causing it to miss its deadline on nearly every job, not just occasionally.

This is why the sibling arm results look sensitive to small conditions rather than smoothly degrading with utilization. It also explains why claim 5's finding matters beyond that one claim: a co-tenant only pushes you over that cliff if it actually inflates your execution time enough to cross it, and whether it does that depends entirely on whether you are competing for the same hardware resource. That is the piece that should let us build a predictive model later: given a real workload's own resource profile, measured directly through standard profiling, we can predict how much margin it needs before this cliff becomes a risk, without having to run every possible workload through the full experimental setup first.

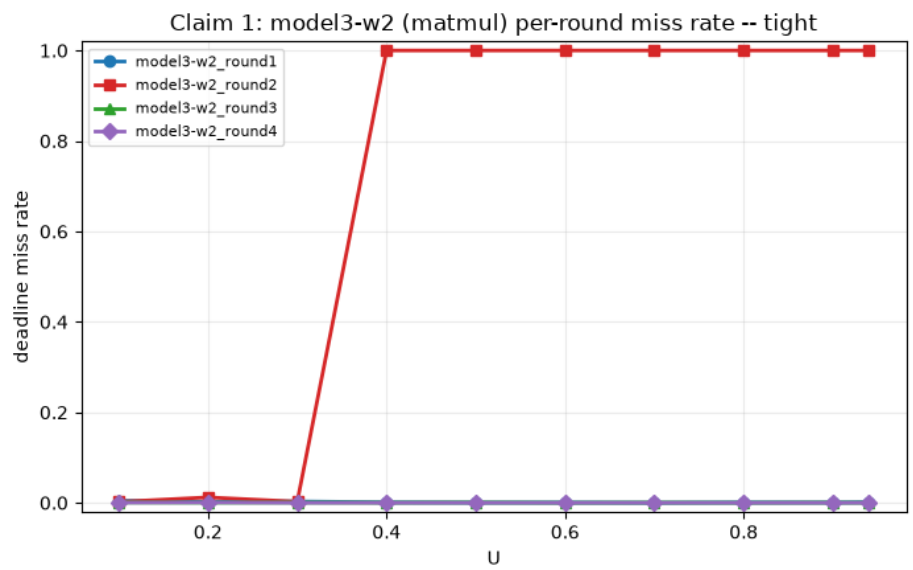
Claim 0: Azure IaaS noise floor on the baseline

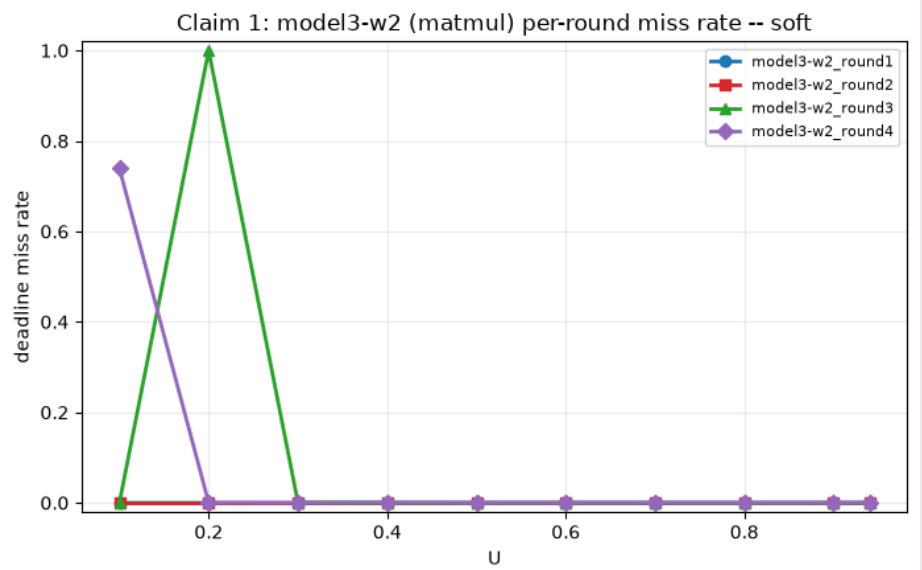




Claim 1: does reservation alone protect against SMT sharing

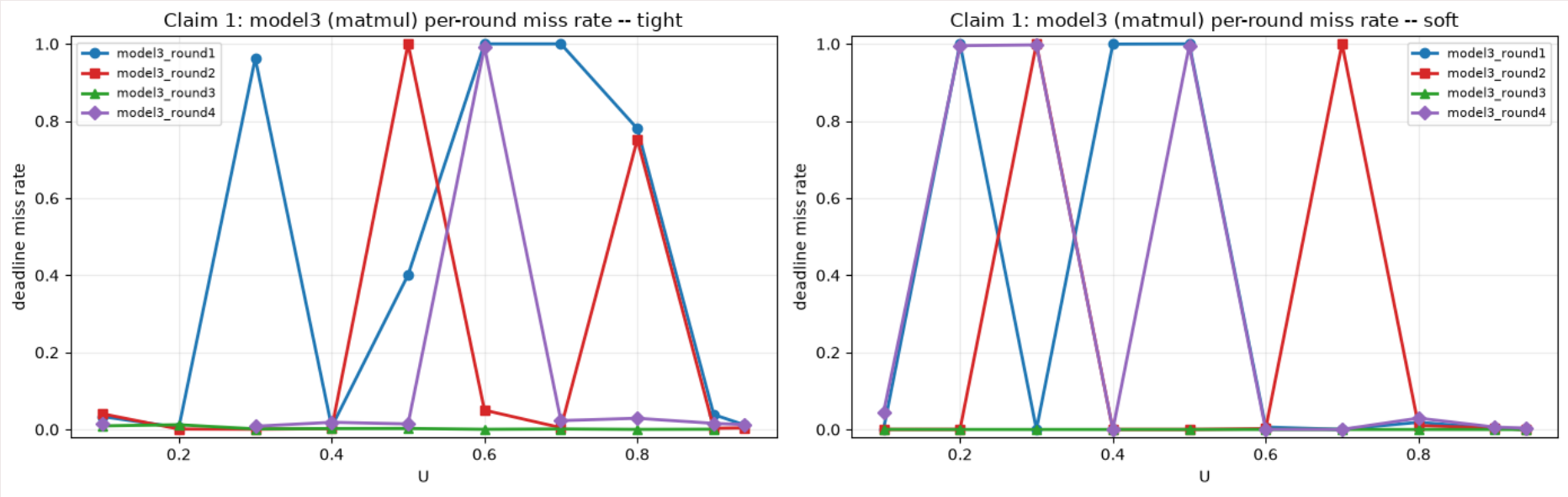
Two containers running on two sibling cores both scheduled with CBS





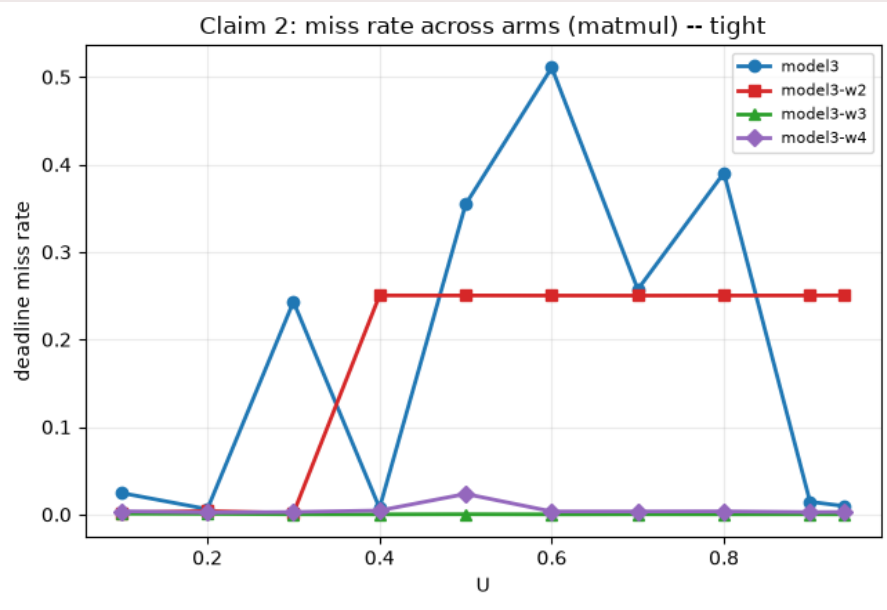
Claim 1

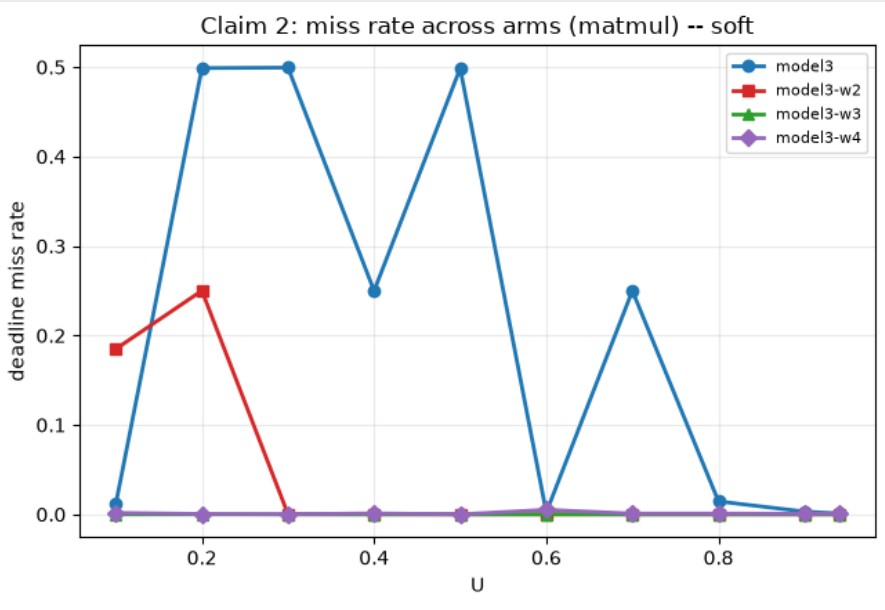
Two containers running on two sibling cores, with one container scheduled with CFS and the other with CBS



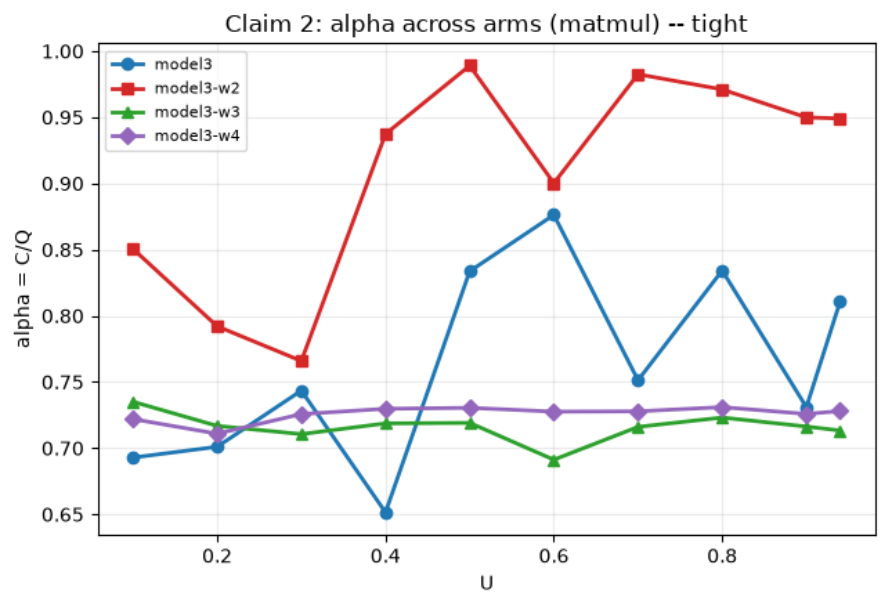
Claim 2: choose physical cores instead of SMT

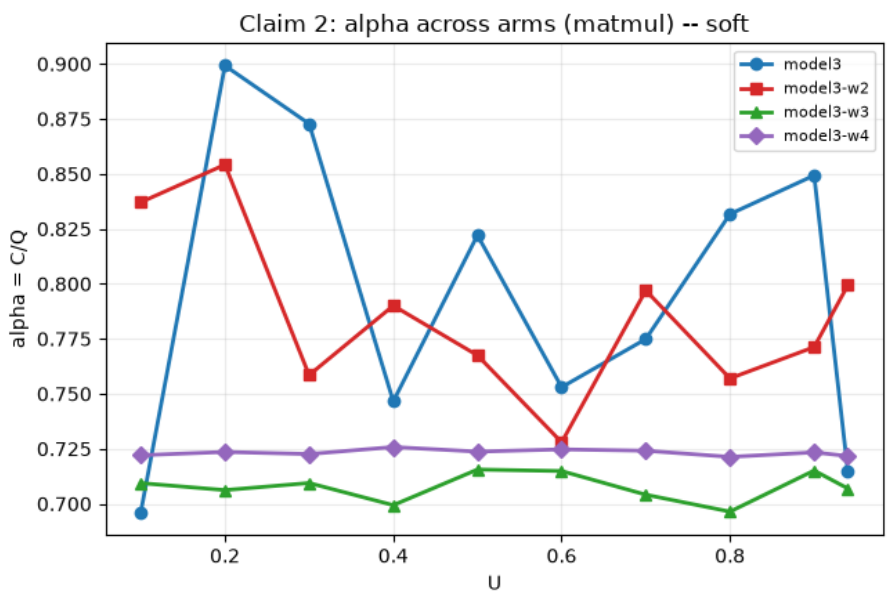
Placing one container (with CFS or CBS) on a different physical core stabilizes miss rate for the other CBS container



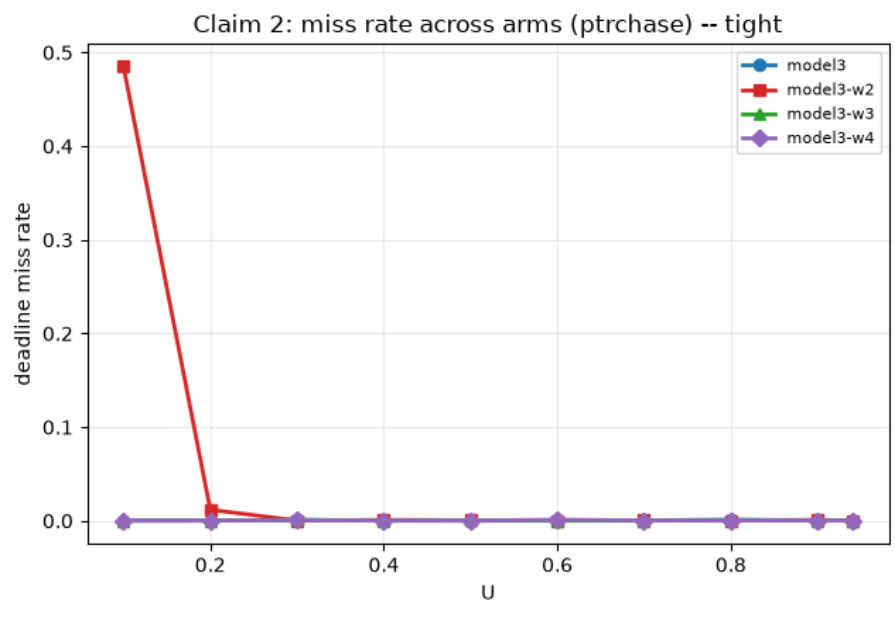


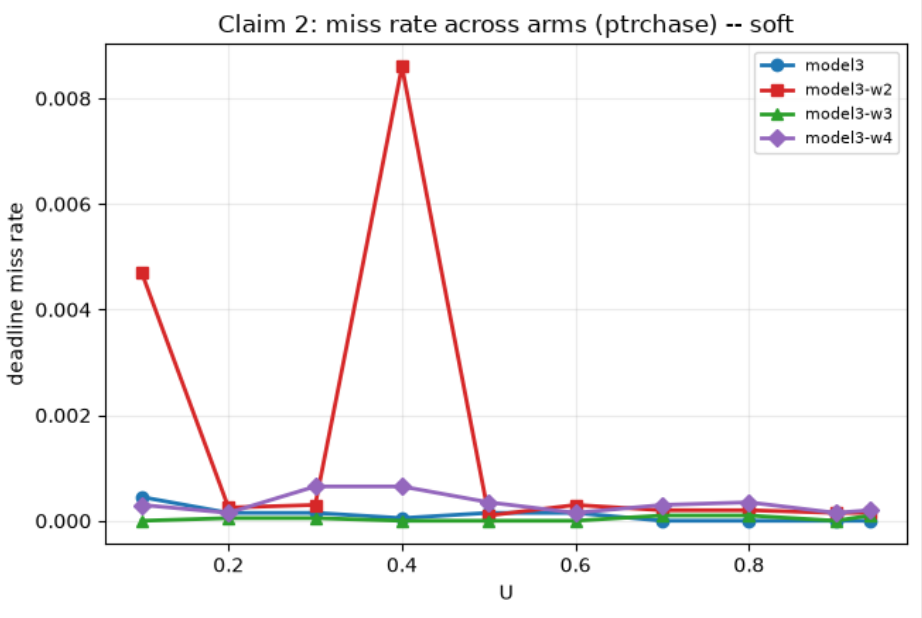
Claim 2 – bandwidth guarantees



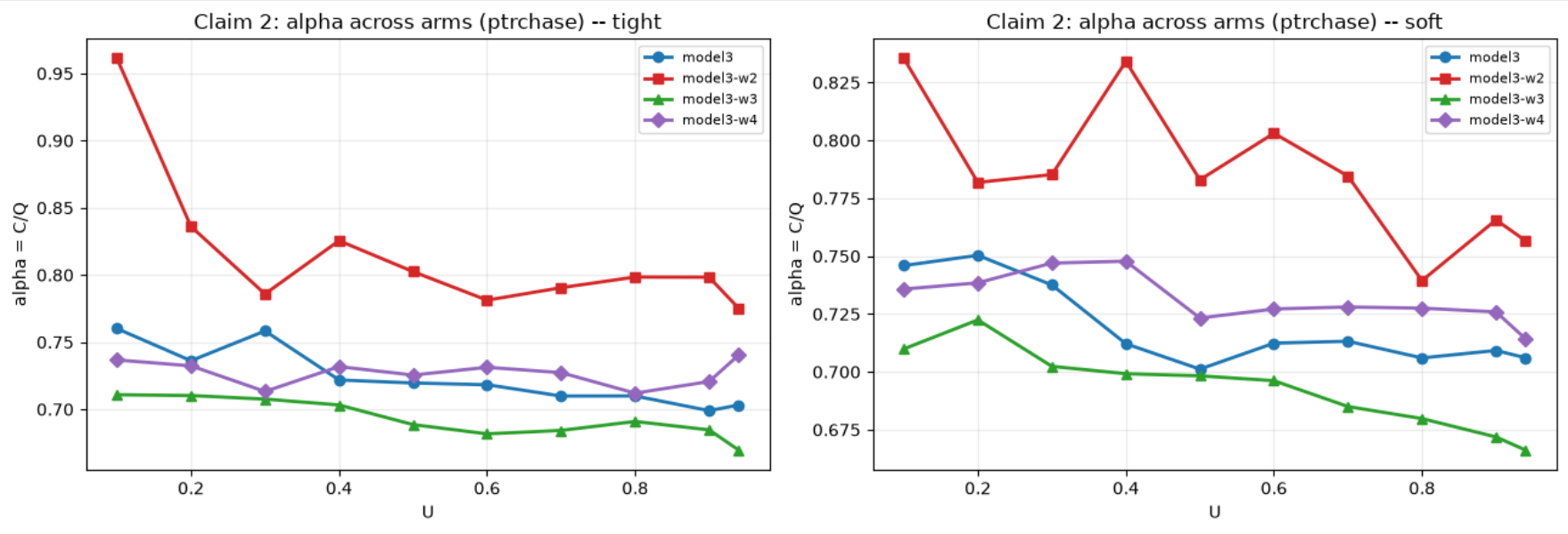


Claim 2 – deadline miss rate





Claim 2 – bandwidth guarantees



Takeaway claim 2:

That is the core of Claim 2 in one comparison: the degradation is not "another workload exists somewhere on the node," it is specifically "sharing a physical core" and it happens whether or not that shared co-tenant is itself well-behaved and properly reserved.

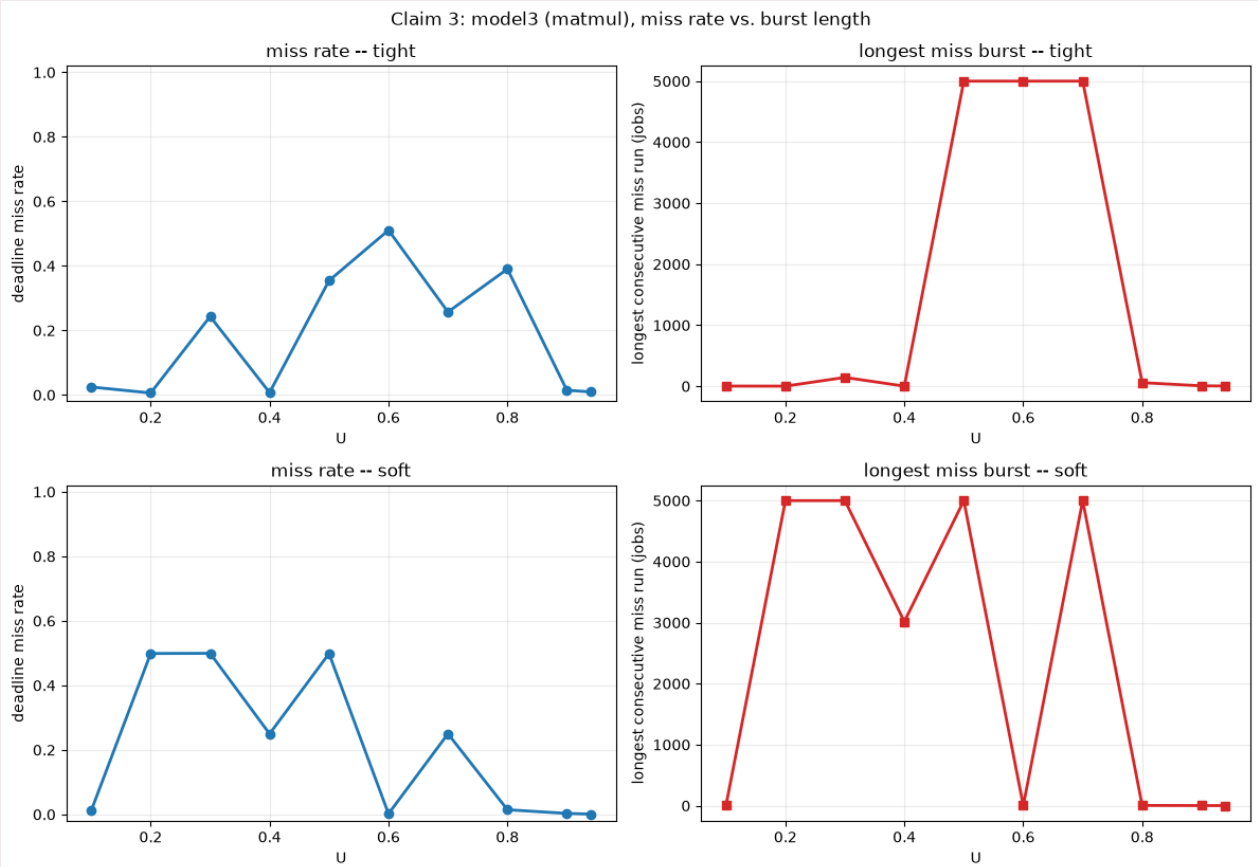
Claim 3:

2 workloads (one CFS and one CBS) running on SMT

A given miss rate could come from scattered isolated misses or from a concentrated burst and these mean very different things in practice.

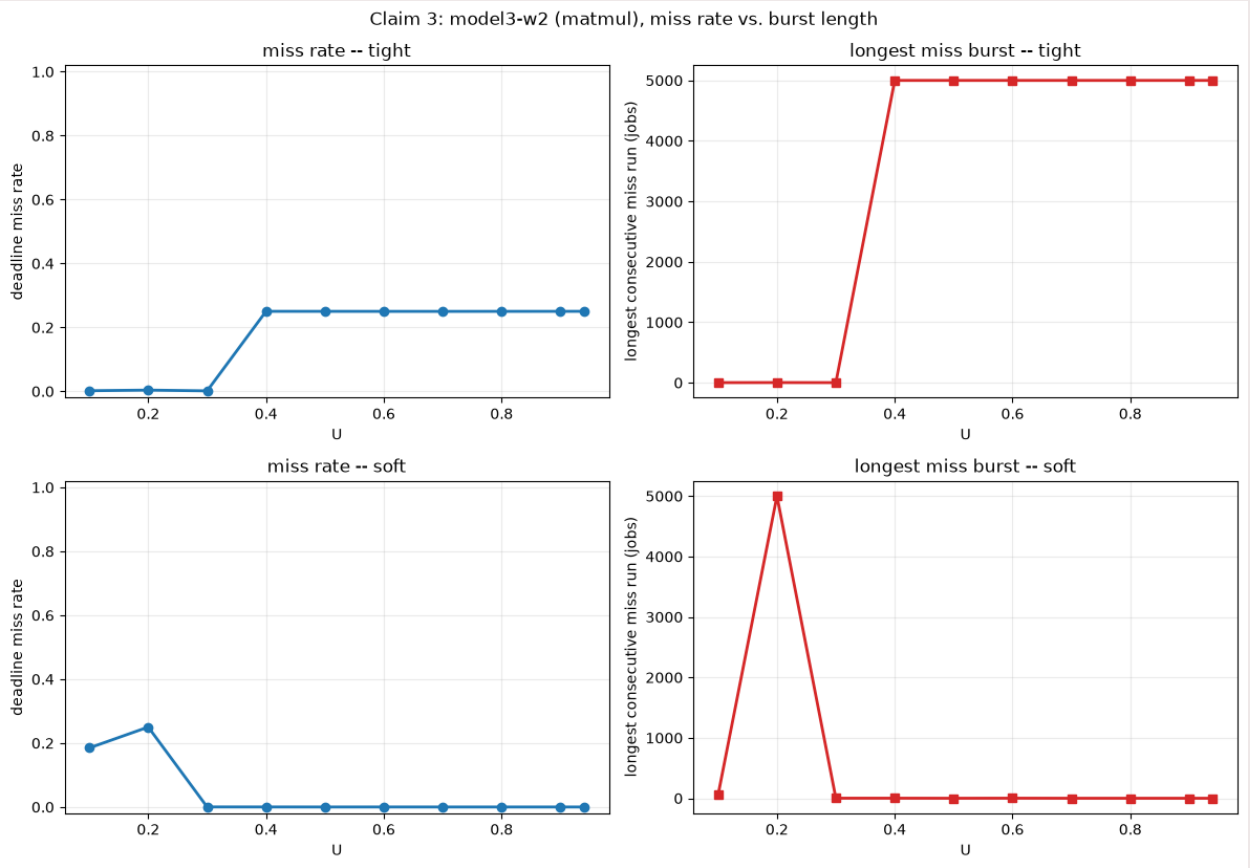
Expectation: misses should cluster specifically at the utilization levels where alpha crosses 1 (the H-CBS throttling cliff), not be spread randomly across U.

Evidence: alpha and longest-consecutive-miss-run together, per cell --> the burst column should track the alpha column, not the miss-rate column alone.



Claim 3

Every cell where the pooled miss rate crosses roughly 25 percent or higher also shows `longest_consecutive_miss_run` at or near 5000, meaning once the throttling cliff triggers, essentially every remaining job in that cell misses in one unbroken run, not scattered isolated misses. Compare model3 soft U0.6 (miss 0.2 percent, longest run 2) against soft U0.2 (miss 49.9 percent, longest run 5000, the entire cell) at nearly the same alpha -- the difference between "occasional miss" and "total lockout" is a small shift in alpha, not a gradual one.



Claim 4

Run on SMT with (CBS, CFS) workloads

=== model3 (matmul) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.696	0.94
1	tight	0.296	0.10

=== model3 (ptrchase) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.940
1	tight	0.94	0.777

Run on SMT with (CBS, CBS) workloads

=== model3-w2 (matmul) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.940	0.94
1	tight	0.388	0.10

=== model3-w2 (ptrchase) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.395
1	tight	0.94	0.940

Run on physical with (CBS, CFS) workloads

=== model3-w3 (matmul) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.94
1	tight	0.94	0.94

=== model3-w3 (ptrchase) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.940
1	tight	0.94	0.288

Claim 4:

Run on physical with (CBS, CFS) workloads

```
=== model3-w4 (matmul) ===
```

	scale	U_safe_p99	U_safe_p999
0	soft	0.940	0.399
1	tight	0.493	0.100

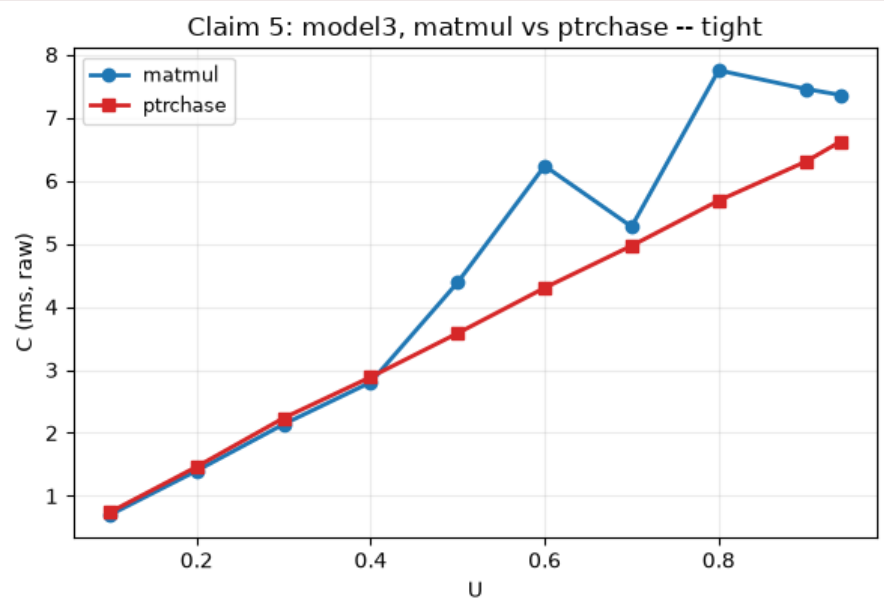
```
=== model3-w4 (ptrchase) ===
```

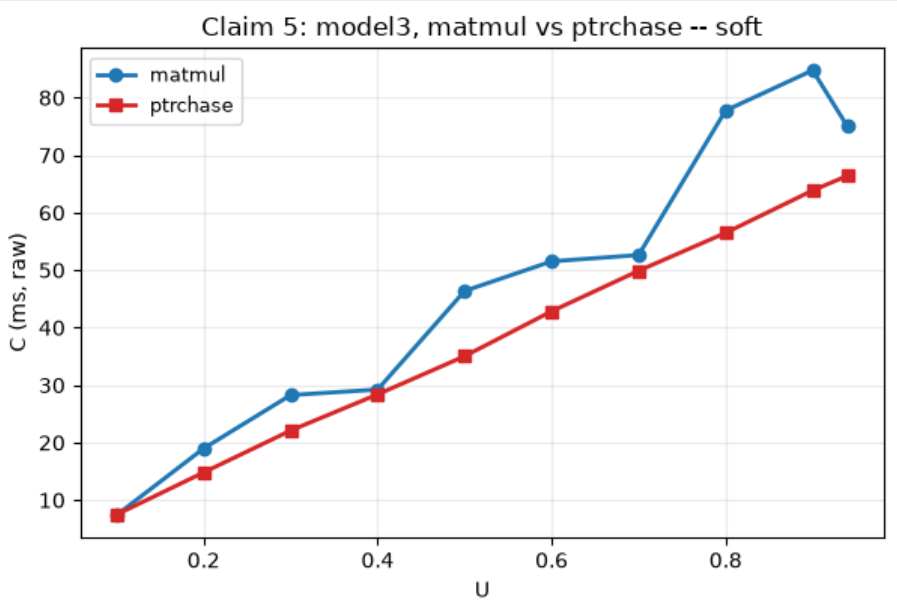
	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.940
1	tight	0.94	0.572

Interpretation: Model3-w3 (both kinds) and the soft scale of model3-w4 sit at the ceiling, U_safe of 0.94, essentially the full tested range. model3-w4's tight scale shows a lower number (p99 0.493, p999 0.100) that looks like a contradiction, but cross-checking against the miss rates in Claim 2's table, model3-w4 tight never exceeds 2.35 percent miss at any single cell.
The explantion is that the p99/p999 metric is a strict standard: it flags the U where even a small, roughly-noise-floor-sized tail probability first appears, not where the guarantee catastrophically fails.

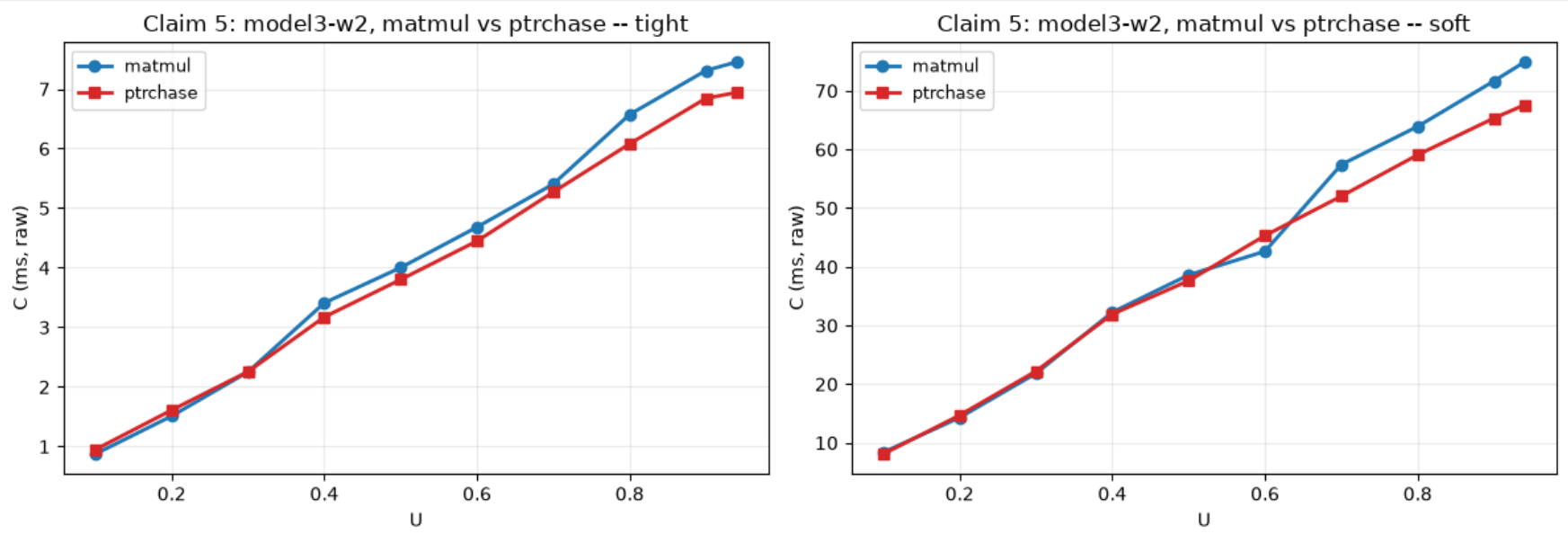
So physical placement keeps failures at the noise floor across the entire range, while sibling placement, when it breaks, breaks by one to two orders of magnitude more, not just a stricter percentile catching the same small effect.

Claim 5: does the workload's own resource profile matter





Claim 5


Claim 5: model3-w2, matmul vs ptrchase -- tight
matmul
ptrchase
C (ms, raw)
U
1
2
3
4
5
6
7
0.2
0.4
0.6
0.8
Claim 5: model3-w2, matmul vs ptrchase -- soft
matmul
ptrchase
C (ms, raw)
U
10
20
30
40
50
60
70
0.2
0.4
0.6
0.8

Claim 5

Interpretation: confirmed, with the magnitude more moderate than the earlier narrower check suggested

The physical arms (model3-w3, model3-w4) hold the ratio at 0.95 to 1.03 across every single cell, both scales. The sibling arms show real, consistent departure from 1: model3 (unreserved competitor) sits at 0.69 to 0.80 specifically at the cells where matmul's own miss rate is elevated (soft U0.2/U0.3/U0.5, tight U0.6/U0.8), meaning ptrchase's execution time stays 20 to 30 percent below matmul's under the identical contention setup. model3-w2 (reserved competitor) shows a weaker, less consistent split (0.90 to 1.09) -- its own breakdown is concentrated in one specific anomalous round rather than a steady per-utilization effect, so the kind contrast is less clean there than for the unreserved arm. Net result: the direction of Claim 5 holds everywhere it can be tested, the cleanest evidence is model3's own sibling-unreserved arm, and the earlier "1.8 to 2.1x" figure should be revised down to roughly 1.25 to 1.4x now that the fuller, cross-round-confirmed dataset is in.

New Version of the dra-rt-driver

The new version done for the driver is working properly now and extensively used in the experiments.

This new version introduces a new parameter: RequestedCpus int[]

The user can request specific cores according to its needs and place the reservation (Q, P) in those cores if the admission test allows it.

```
apiVersion: rt.resource.example.com/v1alpha1
kind: RtClaimParameters
metadata:
  namespace: rt-test1
  name: rtclaims0
spec:
  count: 1
  runtime: 100
  period: 1000
  requestedCpus: [1]

---
apiVersion: rt.resource.example.com/v1alpha1
kind: RtClaimParameters
metadata:
  namespace: rt-test1
  name: rtclaims1
spec:
  count: 1
  runtime: 100
  period: 1000
  requestedCpus: [2]
```

Updates on the workloads

There are two workloads for every type of experiment:

1. Matrix multiplier (only computation depenedent)
2. Prime number calculator (data dependent)

What I want to show with this is the following:
"Predictability guarantees that hold for pure computation (matrix mul) degrade under data-dependent workloads (primes), showing that RT-DRA/H-CBS predictability is workload-sensitive, not universal."

Should we consider adapting the research questions?

At the moment the research questions are:

- RQ1: Where do KubeDeadline's scheduling guarantees break down when deployed on AKS?
- RQ2: How to assign parameters of KubeDeadline, such as budget, deadline and period, to time-sensitive tasks in a cloud environment where execution-time characteristics are non-deterministic?
- RQ3: Which Azure IaaS configuration and infrastructure choices most significantly reduce execution-time variability for time-sensitive workloads running under KubeDeadline, and how should these be selected?

Here a proposal:
Under what placement conditions can real-time workloads in cloud virtual machines achieve predictable execution, and where do the limits of that predictability lie?

Periodic vs. event-triggered activation

Claim	Expectation	Results
Event-triggered activation introduces measurable release jitter beyond periodic's baseline, from the cross-thread dispatch/wakeup path itself, predictability depends on how a task is activated, not just where it's placed	Event's dispatch_latency_us distribution is wider and heavier-tailed than periodic's, which stays tight near zero.	

How to show that?

I've introduced a new model to answer the claim:

- **Timer-driven,** "pull" scheduling. A single thread computes its own release schedule ahead of time (precomputed, seed-deterministic) and sleeps to it via clock_nanosleep(TIMER_ABSTIME).

- **Async "push" scheduling**, the Hollywood principle. A separate generator thread fires a trigger at intervals jittered uniformly over [0.5, 1.5)×period (mean = period, deterministic from --seed).

For the event-based model4, the trigger is generated from the same pod in the same VM, so the focus is purely on the OS-level interference.

But what if we extends it and have the trigger in another VM? Then the anlaysis can be extended to a more realistic scenario with a possible external user requesting an RT-service.

Results obtained and claims

Claim: 0 – Iaas Architecture introduces cloud noise

Claim: Even with zero explicit contention, Azure IaaS execution carries a small but non-zero baseline noise floor from hypervisor and OS-level jitter alone.
Expectations: This noise floor should stay small and small specifically relative to the much larger degradation seen once real contention is introduced in later claims
Results: Confirm expecta

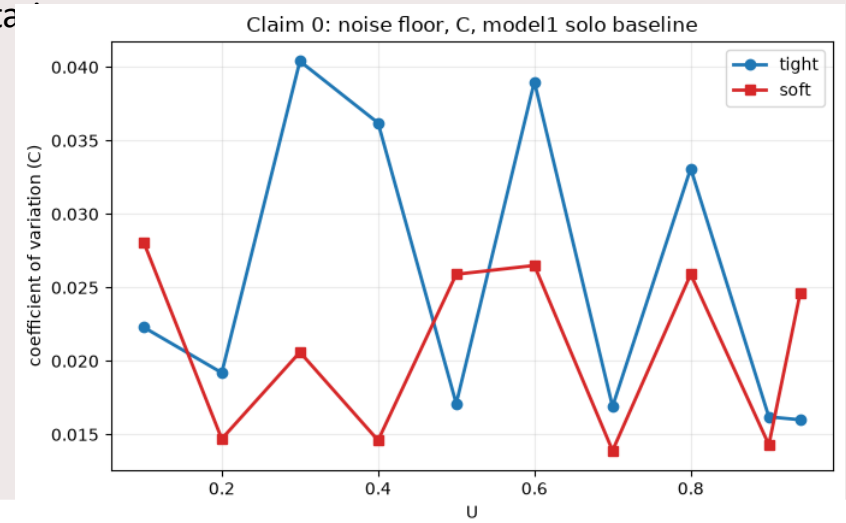
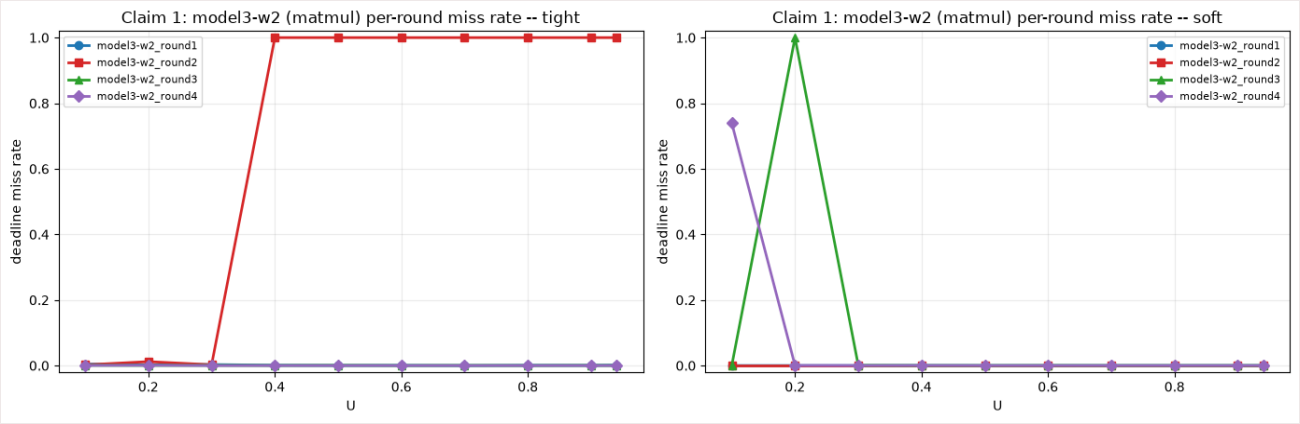


Figure: coefficient of variation (std/mean) of execution time C for the solo baseline (model1), across every utilization on both scales (soft and tight) a flat, low line means stable timing; spikes would mean instability even with nothing else running.

Claim1: Does reservation alone protect against SMT sharing?

Claim: A CPU reservation and passing admission control are not, by themselves, sufficient to protect a real-time task from a properly reserved neighbor if the two share a physical core.
Expectation: Two independently reserved, admission-controlled workloads should still be able to violate each other's guarantees when sharing a physical core, since H-CBS admission accounting operates per logical CPU, not per physical core.



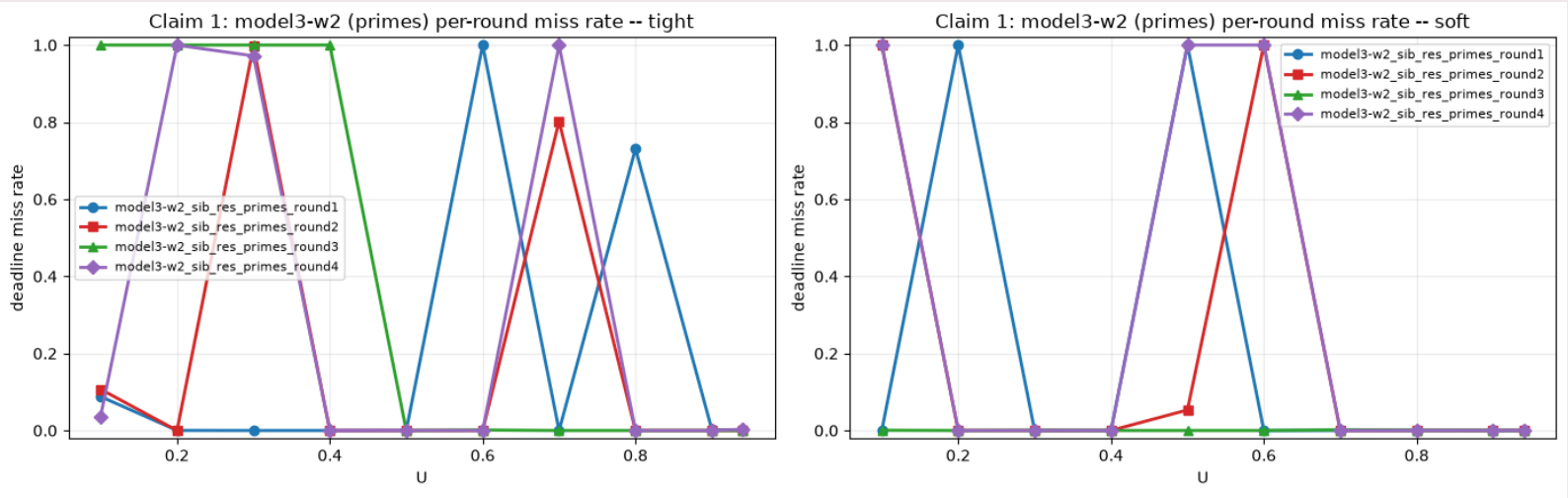
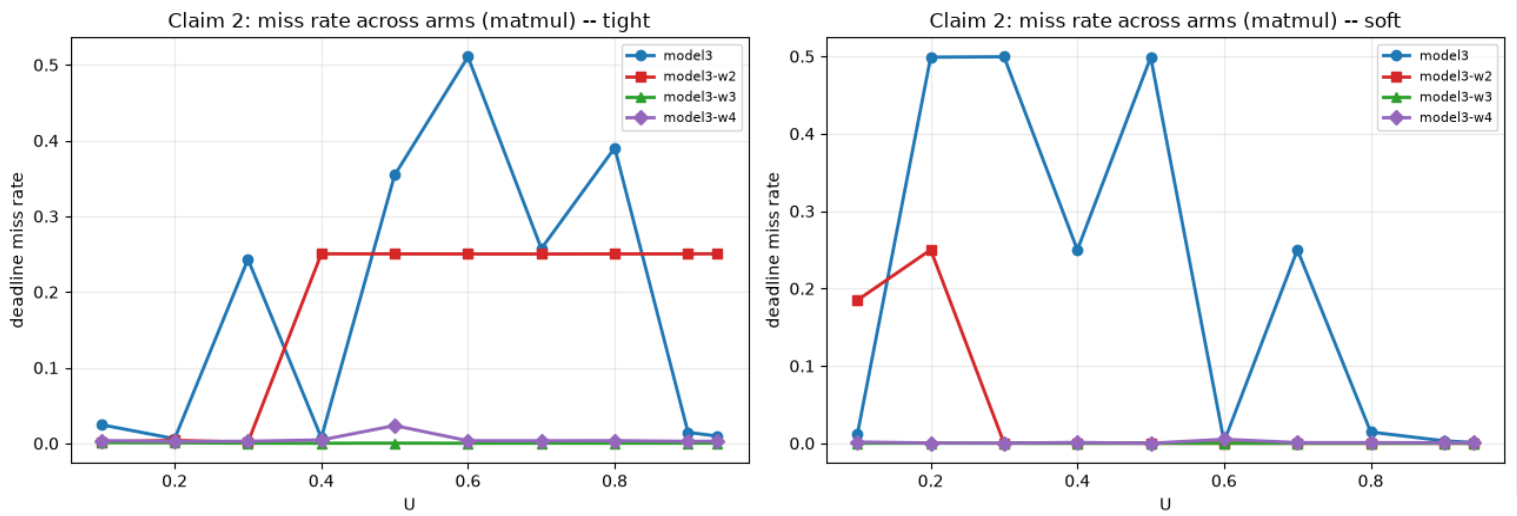


Figure: per-round deadline-miss rate for model3-w2 (sibling placement, reserved competitor), one panel per independently collected round (1–4), darker/redder cells mean a higher fraction of jobs missed their deadline at that utilization.

admission control sizes each reservation's budget Q against a solo-calibrated execution time C. It has no way to know that when the sibling logical CPU is also busy, real C balloons past that calibrated value from shared ALU/execution-port contention.

Claim 2: where exactly does the guarantee break down?

Claim: The predictability guarantee breaks down specifically when a task shares a physical core (SMT) with another workload, not merely when it shares a node with one.
Expectation: Physical-placement arms should stay clean regardless of competitor type; sibling-placement arms should degrade, more severely for an unreserved competitor than a reserved one.



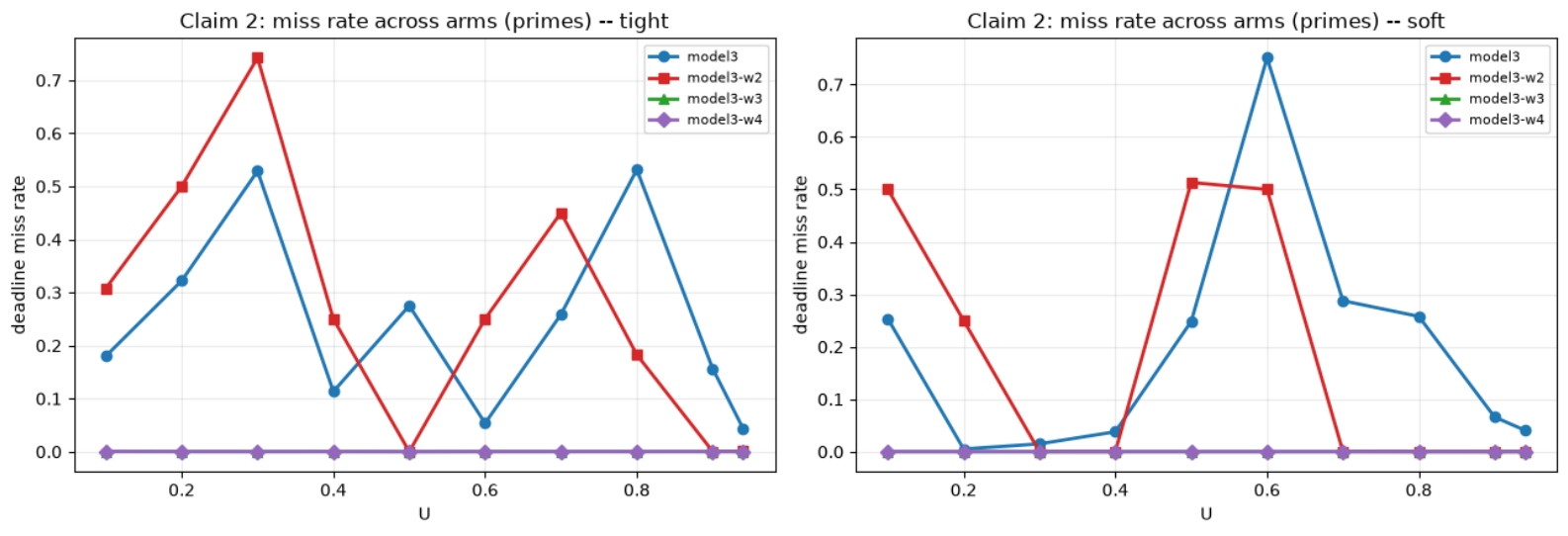
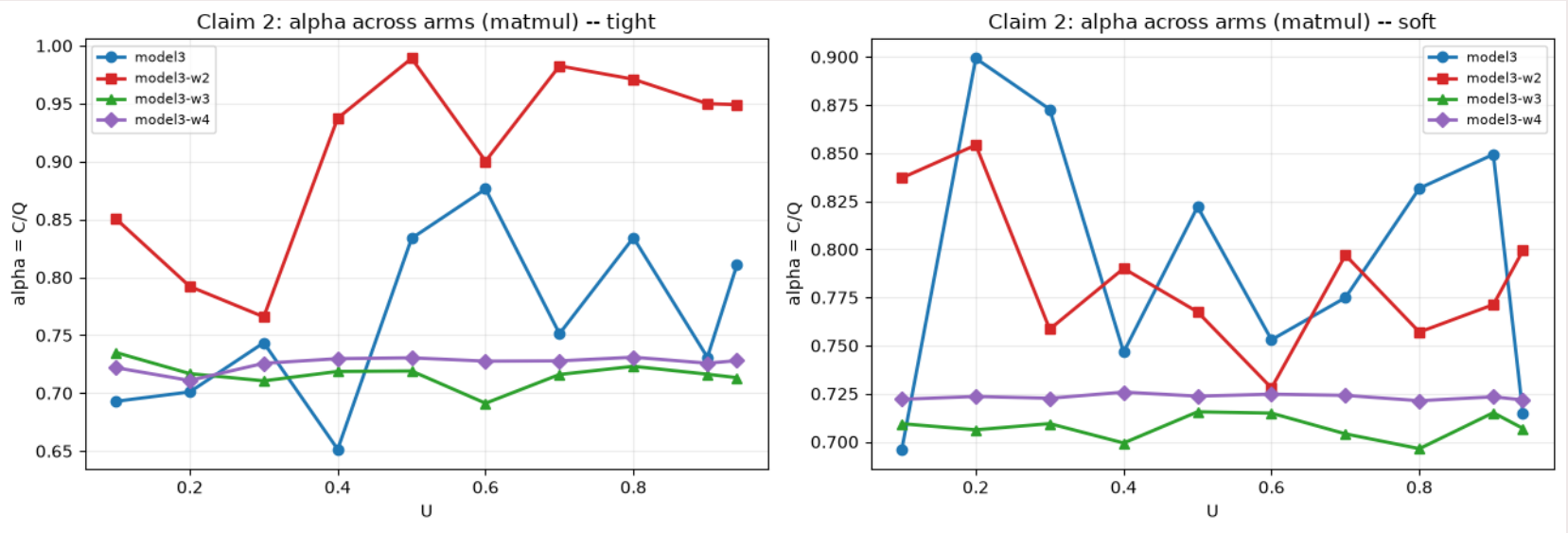
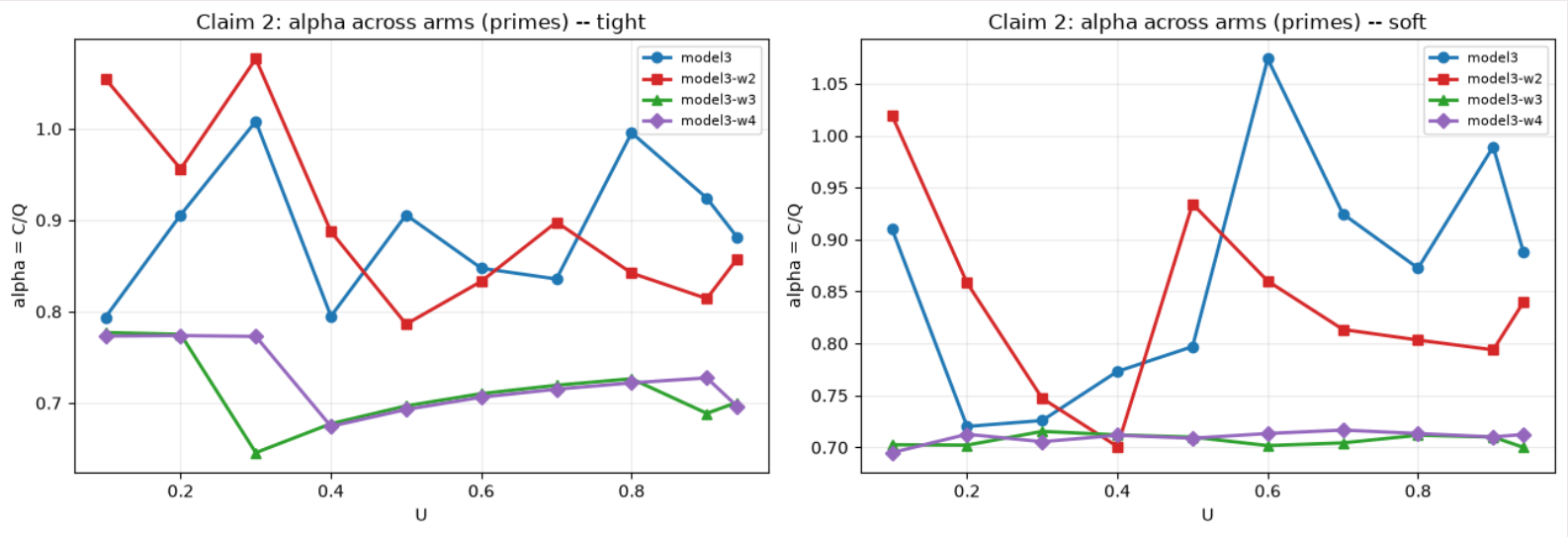


Figure: alpha (C/Q, actual execution time over reserved budget) for all four model3-family arms, plotted across the utilization sweep, a line sitting at or above 1.0 means the reservation is regularly being exceeded.

Confirmed, both kinds :Physical arms (w3/w4) hold alpha **0.65–0.78** regardless of kind or competitor type. Sibling arms (model3/w2) swing to alpha **1.17** (primes). Direct per-job preemption evidence (not just this corelation).

Bandwidth across the models




Claim 2: alpha across arms (primes) -- tight
alpha = C/Q
U
model3
model3-w2
model3-w3
model3-w4
Claim 2: alpha across arms (primes) -- soft
alpha = C/Q
U
model3
model3-w2
model3-w3
model3-w4

Preemption table for the 4 models 3

experimental setup	kind	protected: nonvol_ctxt / mid_preempt	throttled: nonvol_ctxt / mid_preempt
model3 (sibling, unreserved)	matmul	0.001 / 3.1us	1.000 / 34,277us
model3 (sibling, unreserved)	primes	0.001 / 4.5us	1.000 / 21,866us
model3-w2 (sibling, reserved)	matmul	0.000 / 1.3us	1.000 / 18,336us
model3-w2 (sibling, reserved)	primes	0.000 / 1.3us	1.000 / 28,334us
model3-w3 (physical, unreserved)	matmul	0.000 / 1.2us	1.06 / 11,159us (only 34 of 400k jobs)
model3-w3 (physical, unreserved)	primes	0.000 / 1.3us	1.00 / 29,724us (only 1 of 400k jobs)
model3-w4 (physical, reserved)	matmul	0.000 / 1.4us	1.00 / 13,433us (only 1,019 of 400k jobs)
model3-w4 (physical, reserved)	primes	0.000 / 1.1us	1.10 / 17,045us (only 10 of 400k jobs)

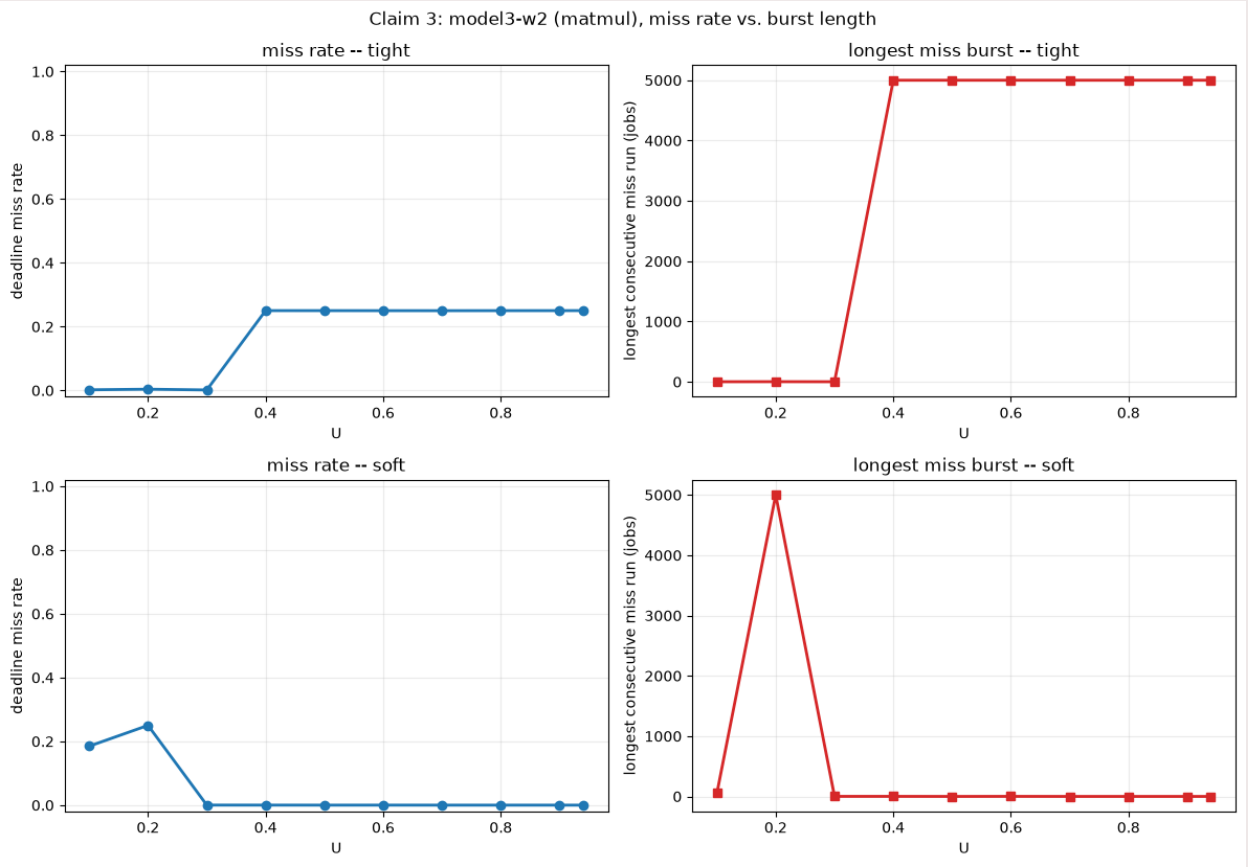
Every target job is split by its own budget ratio alpha = C/Q into protected (alpha<0.85, comfortably under budget) or throttled (alpha≥1.0, budget exceeded). nonvol_ctxt = involuntary context switches per job (the kernel forcibly pulling the thread off-CPU); mid_job_preempt_us = microseconds within that job where it existed but wasn't running, both measured directly by the probe, not inferred from timing math. Protected jobs show ~0 of both in every arm. Throttled jobs jump to ~1 involuntary switch and 11,000-34,000µs lost, the same signature in every arm, sibling or physical. What differs is volume: sibling arms (share one physical core's execution hardware) land tens of thousands of jobs in the throttled bucket; physical arms (separate cores, own hardware) land only a handful out of 400,000. Same mechanism everywhere it can occur, but it can only occur when the pair shares a physical core, evidence this is SMT execution-port contention, not generic scheduling noise.

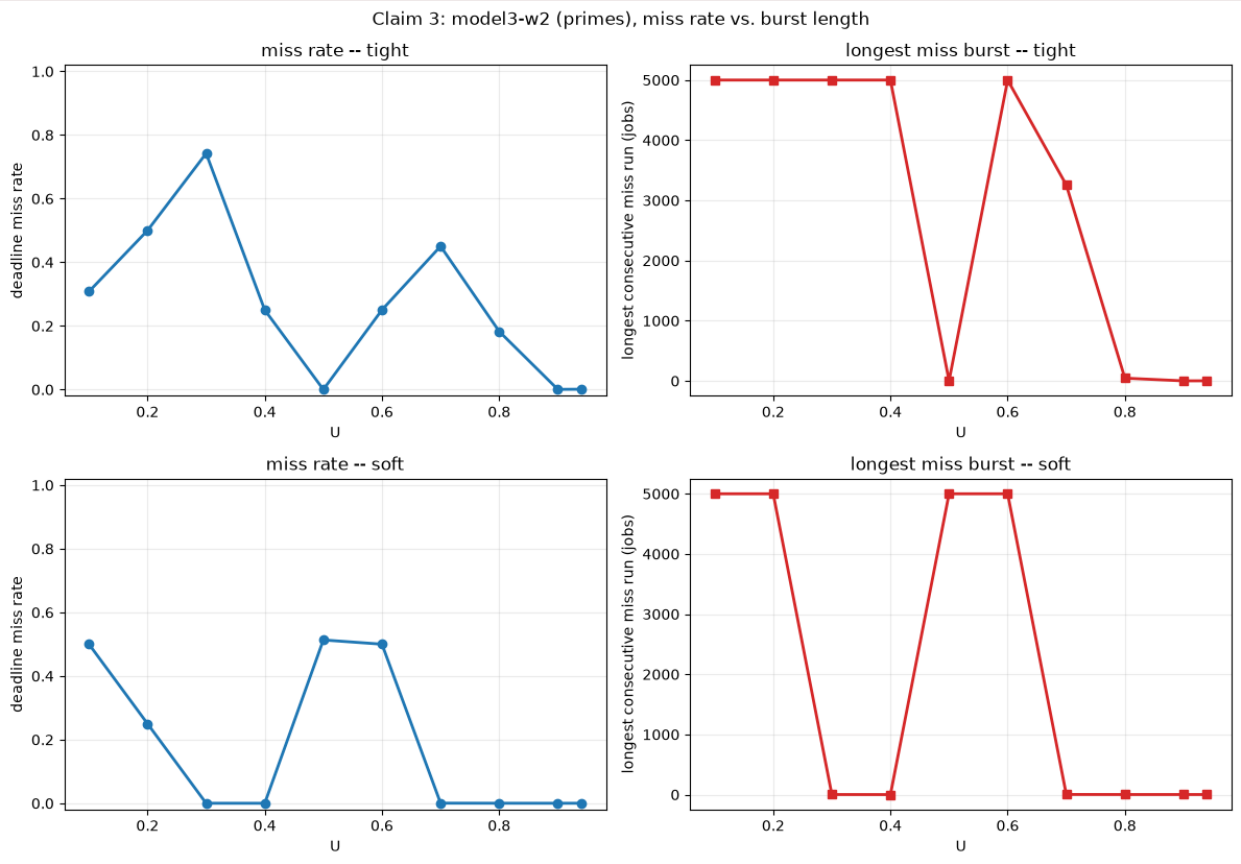
Claim 3: Severity of the miss: burst or scattered misses?

Claim: Once utilization crosses the H-CBS throttling boundary (alpha at or above 1), deadline misses concentrate into one continuous run rather than scattering randomly across jobs.
Expectation: Misses should cluster specifically at the utilization levels where alpha crosses 1, not spread randomly across the utilization sweep.

Figure: alpha and the longest consecutive run of missed deadlines, per cell, model3-w2, a long run near 5000 (the full job count for that cell) means one sustained lockout, not scattered isolated misses.

confirmed, both kinds Every cell above ~25% miss rate also hits a longest-run near 5000 (matmul), total lockout, not scattered misses. Primes: same signature in most cells (7 at longest_run=5000), plus one honest exception (tight U0.7: 45% miss, longest_run only 3257, alpha=0.837<1.0), consistent with primes' own higher per-job variance, not against the mechanism.


Claim 3: model3-w2 (matmul), miss rate vs. burst length
miss rate -- tight
longest miss burst -- tight
miss rate -- soft
longest miss burst -- soft
deadline miss rate
longest consecutive miss run (jobs)
U
0.0
0.2
0.4
0.6
0.8
1.0
0
1000
2000
3000
4000
5000


Claim 3: model3-w2 (primes), miss rate vs. burst length
miss rate -- tight
deadline miss rate
U
longest miss burst -- tight
longest consecutive miss run (jobs)
U
miss rate -- soft
deadline miss rate
U
longest miss burst -- soft
longest consecutive miss run (jobs)
U

Claim 4: Is there a suitable utilization threshold for s?

=== model3 (matmul) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.696	0.94
1	tight	0.296	0.10

=== model3 (primes) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.297	0.1
1	tight	0.100	0.1

=== model3-w2 (matmul) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.940	0.94
1	tight	0.388	0.10

=== model3-w2 (primes) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.494	0.493
1	tight	0.599	0.596

=== model3-w3 (matmul) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.94
1	tight	0.94	0.94

=== model3-w3 (primes) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.94
1	tight	0.94	0.94

=== model3-w4 (matmul) ===

	scale	U_safe_p99	U_safe_p999
0	soft	0.940	0.399
1	tight	0.493	0.100

=== model3-w4 (primes) ===

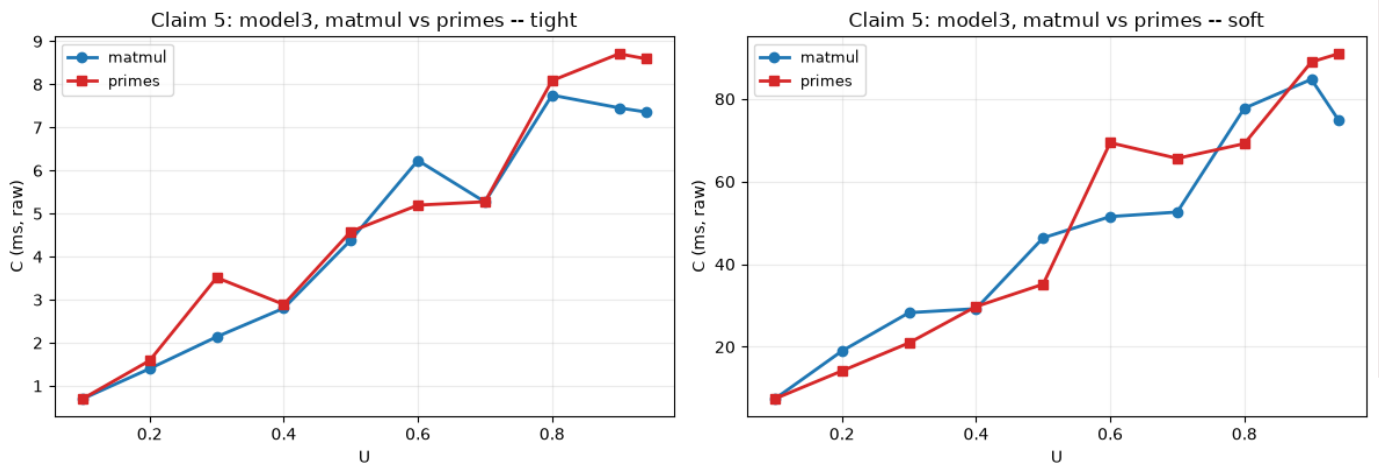
	scale	U_safe_p99	U_safe_p999
0	soft	0.94	0.94
1	tight	0.94	0.94

Claim: There exists a concrete, measurable utilization threshold beyond which a sibling-placed reservation's guarantee can no longer be trusted, while a physically-placed reservation holds across the full tested range.
Expectation: Physical core setup should hold guarantees across the full tested range; sibling core setup should show an identifiable utilization beyond which the guarantee breaks down.

Results: Physical cores setup hold at the ceiling (0.94) in both kinds, no exceptions. Sibling arms drop hard under primes: model3 soft p99 0.696→0.297, tight p99 0.296→0.100. model3-w2 is mixed (soft worse, tight better),flagged as open, needs a 5th round.

Claim 5: Does the workload's resource profile matter?

Claim: The severity of SMT-sharing degradation depends on what kind of computation is actually running on both sides of the shared core, not just on the fact that a physical core is shared at all.
Expectation: A compute-bound pairing (matmul target + matmul competitor, saturating the shared execution ports) should be hurt differently than a data-dependent pairing (primes + primes, more branching/division, a different usage pattern of that same shared pipeline); physical-placement arms, with no sharing at all, should show no such gap either way.



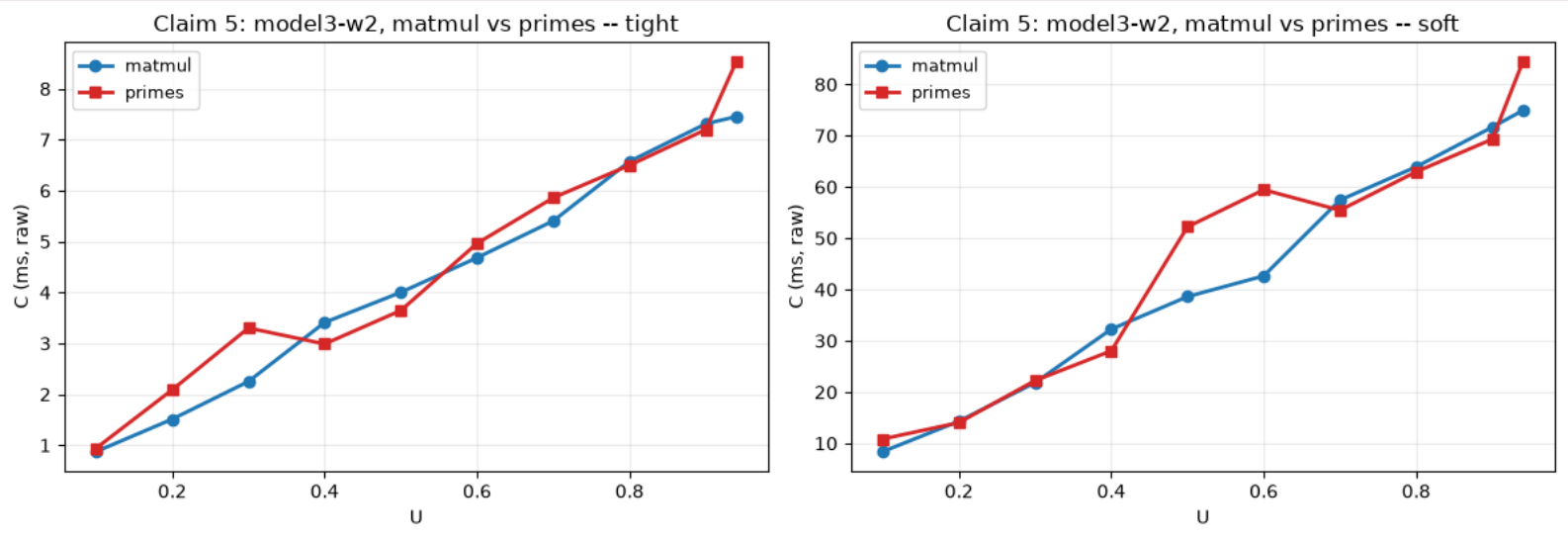


Figure: the target's raw median execution time C (milliseconds), plotted separately for a full matmul-mode sweep and a full primes-mode sweep of the *same* configuration, across utilization. The two curves are *not* expected to sit at the same height. primes and matmul cost different amounts even running solo. What matters is whether each curve inflates by the same *proportion* as utilization (and contention) rises. That proportion is exactly what C_p50_ratio, below, quantifies directly, this figure is the raw data it's computed from.